

Processamento paralelo

- 17.1** Organizações de múltiplos processadores
 - Tipos de sistemas de processadores paralelos
 - Organizações paralelas
- 17.2** Multiprocessadores simétricos
 - Organização
 - Considerações sobre projeto dos sistemas operacionais para multiprocessadores
 - Um mainframe SMP
- 17.3** Coerência de cache e protocolo MESI
 - Soluções por software
 - Soluções por hardware
 - O protocolo MESI
- 17.4** *Multithreading* e chips multiprocessadores
 - *Multithreading* implícito e explícito
 - Abordagens para *multithreading* explícito
 - Exemplos de sistemas
- 17.5** *Clusters*
 - Configurações de *cluster*
 - Questões sobre projeto dos sistemas operacionais
 - Arquitetura de um *cluster* computacional
 - Servidores blade
 - *Clusters* comparados a SMP
- 17.6** Acesso não uniforme à memória
 - Motivação
 - Organização
 - Prós e contras de NUMA
- 17.7** Computação vetorial
 - Abordagens para computação vetorial
 - Recurso vetorial do IBM 3090
- 17.8** Leitura recomendada e sites Web
 - Sites Web recomendados

PRINCIPAIS PONTOS

- Um jeito tradicional para melhorar o desempenho do sistema é usar múltiplos processadores que possam executar em paralelo para suportar uma certa carga de trabalho. Duas organizações mais comuns de múltiplos processadores são **multiprocessadores simétricos** (SMP, do inglês *symmetric multiprocessor*) e *clusters*. Mais recentemente, sistemas de **acesso não uniforme à memória** (NUMA, do inglês *nonuniform memory access*) foram introduzidos comercialmente.
- Um SMP consiste de vários processadores semelhantes dentro de um mesmo computador, interconectados por um barramento ou algum tipo de arranjo de comutação. O problema mais crítico a ser resolvido em um SMP é a coerência de cache. Cada processador possui a sua própria cache e, assim, é possível que uma determinada informação esteja presente em mais de uma cache. Se tal informação for alterada em uma cache, então a memória principal e a outra cache possuem uma versão inválida dessa informação. Os protocolos de coerência de cache são projetados para lidar com esse problema.
- Quando mais de um processador é implementado em um chip único, a configuração é conhecida como **chip de multiprocessamento**. Um esquema de projeto relacionado é replicar alguns dos componentes de um único processador para que o processador possa executar várias threads de forma concorrente; isto é conhecido como um **processador multithread**.
- Um **cluster** é um grupo de computadores completo conectados trabalhando juntos como um recurso computacional unificado que pode criar a ilusão de ser apenas uma máquina. O termo *computador completo* significa um sistema que pode funcionar por conta própria, separado do *cluster*.
- Um sistema NUMA é um multiprocessador de memória compartilhada em que o tempo de acesso para determinado processador a uma palavra na memória varia de acordo com a posição da palavra na memória.
- Um propósito especial de organização paralela é o recurso vetorial, o qual é dedicado ao processamento de vetores ou matrizes de dados.

Tradicionalmente, o computador tem sido visto como uma máquina sequencial. A maioria das linguagens de programação de computadores requer que o programador especifique algoritmos como uma sequência de instruções. Os processadores executam programas executando as instruções de máquina em sequência e uma por vez. Cada instrução é executada em uma sequência de operações (obter instrução, obter operandos, executar operação, armazenar resultados).

Esta visão do computador nunca foi totalmente verdadeira. Em nível de micro-operações, vários sinais de controle são gerados ao mesmo tempo. O pipeline de instruções, pelo menos quando há sobreposição de operações de leitura e execução, está presente há muito tempo. Ambos são exemplos de desempenho de funções em paralelo. Esta abordagem é aprofundada com a organização superescalar, a qual explora paralelismo em nível de instruções. Em uma máquina superescalar existem várias unidades de execução dentro de um único processador e estas podem executar várias instruções de um mesmo programa em paralelo.

À medida que a tecnologia computacional evoluiu e o custo de hardware computacional baixou, os projetistas procuraram mais e mais oportunidades para paralelismo, normalmente para melhorar o desempenho e, em alguns casos, para aumentar a disponibilidade. Depois de uma introdução, este capítulo analisa algumas abordagens mais promissoras para organização paralela. Primeiro, analisamos multiprocessadores simétricos (SMP), um dos primeiros e ainda mais comuns exemplos da organização paralela. Em uma organização SMP, vários processadores compartilham uma memória comum. Esta organização levanta a questão da coerência de cache, a qual uma seção separada é dedicada. Depois descrevemos os *clusters*, os quais consistem em vários computadores independentes organizados de forma cooperativa. A seguir, o capítulo analisa os processadores multithread e chips multiprocessadores. *Clusters* tornaram-se muito comuns para suportar cargas de trabalho que estão além da capacidade de um único SMP. Outra abordagem para uso de vários processadores que analisamos são máquinas de acesso não uniforme à memória (NUMA). A abordagem NUMA é relativamente nova e ainda não aprovada no mercado, mas é frequentemente considerada como uma alternativa para abordagem SMP ou *cluster*. Finalmente, este capítulo analisa as abordagens de organização de hardware para computação vetorial. Estas abordagens otimizam a ALU para processamento de vetores ou matrizes de números de ponto flutuante. Elas são comuns na classe de sistemas conhecida como *supercomputadores*.



17.1 Organizações de múltiplos processadores



Tipos de sistemas de processadores paralelos

Uma taxonomia introduzida inicialmente por Flynn (FLYNN, 1972^a) é ainda a maneira mais comum de categorizar sistemas com capacidade de processamento paralelo. Flynn propôs as seguintes categorias de sistemas computacionais:

- **Instrução única, único dado (SISD, do inglês *single instruction, single data*):** um processador único executa uma única sequência de instruções para operar nos dados armazenados em uma única memória. Uniprocessadores enquadram-se nesta categoria.
- **Instrução única, múltiplos dados (SIMD, do inglês *single instruction, multiple data*):** uma única instrução de máquina controla a execução simultânea de uma série de elementos de processamento em operações básicas. Cada elemento de processamento possui uma memória de dados associada, então cada instrução é executada em um conjunto diferente de dados por processadores diferentes. Processadores de vetores e matrizes se enquadram nesta categoria e são discutidos na Seção 18.7.
- **Múltiplas instruções, único dado (MISD, do inglês *multiple instruction, single data*):** uma sequência de dados é transmitida para um conjunto de processadores, onde cada um executa uma sequência de instruções diferente. Esta estrutura não é implementada comercialmente.
- **Múltiplas instruções, múltiplos dados (MIMD, do inglês *multiple instruction, multiple data*):** Um conjunto de processadores que executam sequências de instruções diferentes simultaneamente em diferentes conjuntos de dados. SMPs, *clusters* e sistemas NUMA enquadram-se nesta categoria.

Com a organização MIMD, os processadores são de uso geral; cada um é capaz de processar todas as instruções necessárias para efetuar transformação de dados apropriada. MIMDs podem ser ainda divididos pelos meios de comunicação do processador (Figura 17.1). Se os processadores compartilham uma memória comum, então cada

processador acessa programas e dados armazenados na memória compartilhada e os processadores se comunicam uns com os outros por meio dessa memória. A forma mais comum desse sistema é conhecida como **multiprocessador simétrico (SMP)**, o qual examinamos na Seção 17.2. Em um SMP, múltiplos processadores compartilham uma única memória ou um pool de memória por um barramento compartilhado ou algum outro mecanismo de interconexão; um recurso diferenciado é que o tempo de acesso à memória de qualquer região de memória é aproximadamente o mesmo para cada processador. Um desenvolvimento mais recente é a organização de **acesso não uniforme à memória (NUMA)**, a qual é descrita na Seção 17.5. Como o próprio nome sugere, o tempo de acesso à memória de diferentes regiões da memória pode diferir para um processador NUMA.

Uma coleção de uniprocessadores independentes ou SMPs pode ser interconectada para formar um **cluster**. A comunicação entre os computadores é feita por caminhos fixos ou por alguma facilidade de rede.



Organizações paralelas

A Figura 17.2 ilustra a organização geral da taxonomia da Figura 17.1. A Figura 17.2a mostra a estrutura de um SISD. Existe um tipo de unidade de controle (CU, do inglês *control unit*) que fornece um fluxo de instruções (IS, do inglês *instruction stream*) para a unidade de processamento (PU, do inglês *processing unit*). A unidade de processamento opera em cima de um único fluxo de dados (DS, do inglês *data stream*) de uma unidade de memória (MU, do inglês *memory unit*). Com um SIMD, ainda há uma única unidade de controle, alimentando agora um único fluxo de instruções para várias PUs. Cada PU pode ter a sua própria memória dedicada (Figura 17.2b) ou pode haver uma memória compartilhada. Finalmente, com MIMD, há várias unidades de controle, cada uma alimentando um fluxo de instruções separado para a sua própria PU. O MIMD pode ser um multiprocessador de memória compartilhada (Figura 17.2c) ou um computador de memória distribuída (Figura 17.2d).

As questões de projeto relativas a SMPs, *clusters* e NUMA são complexas e envolvem pontos de organização física, estruturas de interconexão, comunicação entre processadores, projeto de sistemas operacionais e técnicas de aplicações de software. O nosso foco aqui é, em primeiro lugar, a organização, embora analisamos brevemente as questões sobre projeto de sistemas operacionais.

Figura 17.1 Uma taxonomia de arquiteturas de processadores paralelos

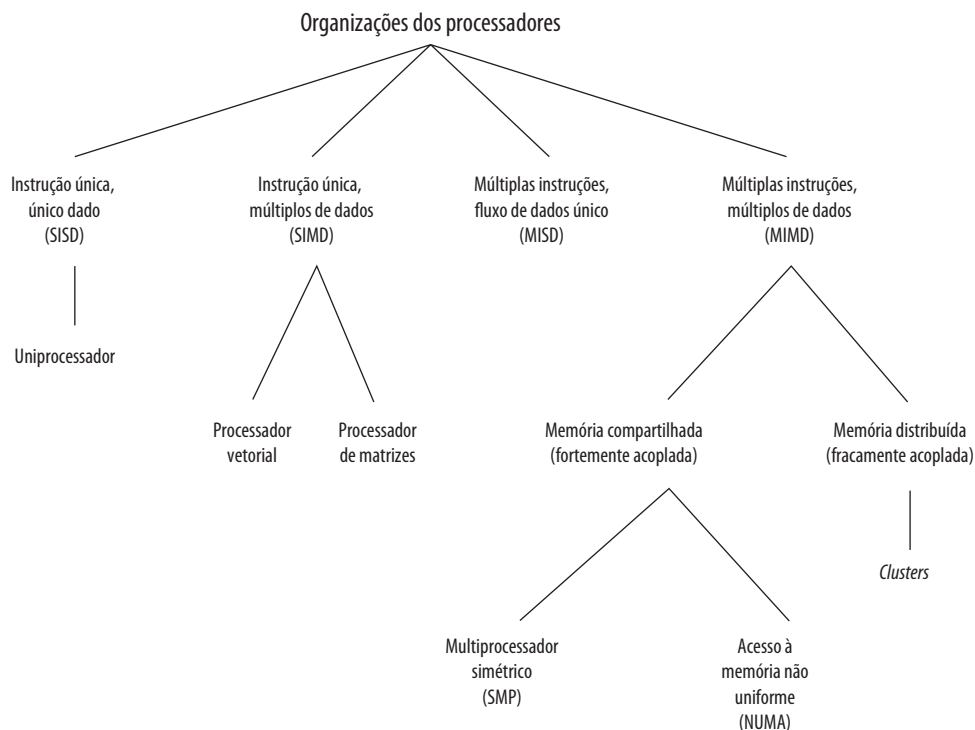
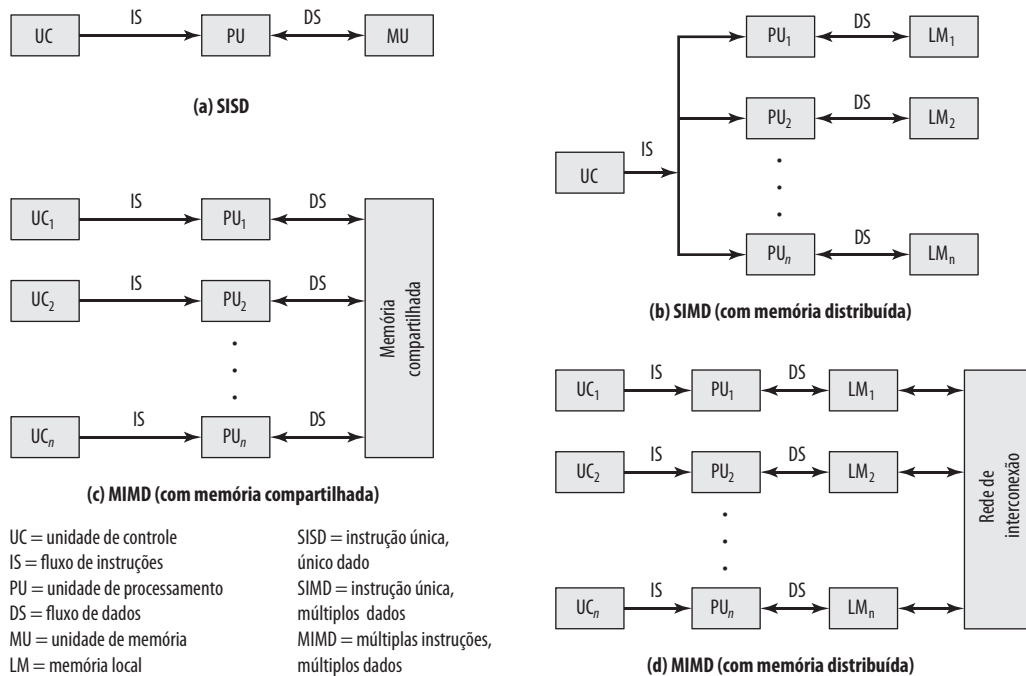


Figura 17.2 Organizações alternativas de computadores

17.2 Multiprocessadores simétricos

Até recentemente, quase todos os computadores pessoais e a maioria de estações de trabalho continham um único microprocessador de propósito geral. À medida que a demanda por desempenho aumenta e o custo de microprocessadores continua a baixar, os fabricantes têm introduzido sistemas com uma organização SMP. O termo *SMP* refere-se a uma arquitetura de hardware computacional e também ao comportamento do sistema operacional que reflete essa arquitetura. Um SMP pode ser definido como um sistema de computação independente com as seguintes características:

1. Há dois ou mais processadores semelhantes de capacidade comparável.
2. Esses processadores compartilham a mesma memória principal e os recursos de E/S, e são interconectados por um barramento ou algum outro esquema de conexão interna, de tal forma que o tempo de acesso à memória é aproximadamente igual para cada processador.
3. Todos os processadores compartilham acesso aos dispositivos de E/S, ou pelos mesmos canais ou por canais diferentes que fornecem caminhos para o mesmo dispositivo.
4. Todos os processadores desempenham as mesmas funções (daí o termo *simétrico*).
5. O sistema é controlado por um sistema operacional integrado que fornece interação entre processadores e seus programas em nível de trabalhos, tarefas, arquivos ou elementos de dados.

Os itens de 1 a 4 são autoexplicativos. O item 5 ilustra um dos contrastes com um sistema de multiprocessamento fracamente acoplado, como um *cluster*. No último, a unidade física de interação é normalmente uma mensagem ou um arquivo completo. Em um SMP, elementos individuais de dados podem constituir o nível de interação e pode haver um alto grau de cooperação entre processos.

O sistema operacional de um SMP faz o agendamento de processos ou threads por meio de todos os processadores. Uma organização SMP possui um número de vantagens potenciais em relação a uma organização de uniprocessador, incluindo o seguinte:

- **Desempenho:** se o trabalho a ser feito por um computador pode ser organizado de tal forma que algumas partes do trabalho possam ser feitas em paralelo, então um sistema com vários processadores vai atingir desempenho melhor do que um com um único processador do mesmo tipo (Figura 17.3).
- **Disponibilidade:** em um multiprocessador simétrico, como todos os processadores podem efetuar as mesmas funções, a falha de um único processador não trava a máquina. Em vez disso, o sistema pode continuar a funcionar com desempenho reduzido.
- **Crescimento incremental:** o usuário pode melhorar o desempenho de um sistema acrescentando um processador adicional.
- **Escalabilidade:** fornecedores podem oferecer uma série de produtos com diferentes preços e características de desempenho com base no número de processadores configurado no sistema.

É importante observar que estes benefícios são potenciais e não garantidos. O sistema operacional deve fornecer ferramentas e funções para explorar o paralelismo em um sistema SMP.

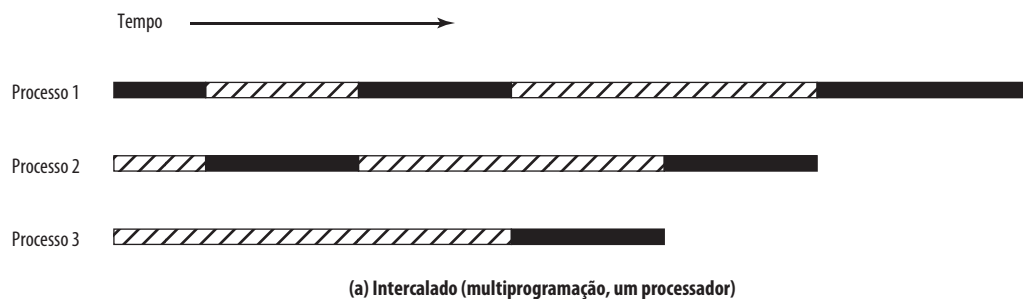
Um recurso atraente de um SMP é que a existência de vários processadores é transparente para usuário. O sistema operacional toma conta do escalonamento de threads ou processos em processadores individuais e da sincronização entre processadores.



Organização

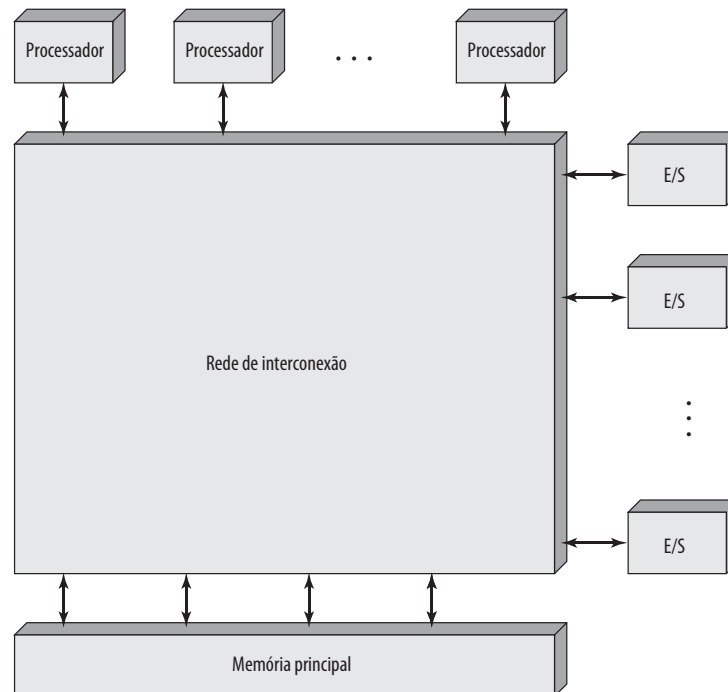
A Figura 17.4 ilustra, em termos gerais, a organização de um sistema multiprocessado. Existem dois ou mais processadores. Cada um é autossuficiente, incluindo uma unidade de controle, uma ALU, registradores e, normalmente, um ou mais níveis de cache. Cada processador possui acesso à memória principal compartilhada e aos dispositivos de E/S por meio de alguma forma de mecanismo de interconexão. Os processadores podem comunicar-se uns com outros pela memória (mensagens e informações de estado são colocadas em áreas comuns da memória). Os processadores também podem trocar sinais diretamente. A memória é frequentemente

Figura 17.3 Multiprogramação e multiprocessamento



 Bloqueado

 Executando

Figura 17.4 Diagrama de blocos genérico de um multiprocessador fortemente acoplado

organizada de tal forma que vários acessos simultâneos a blocos de memória separados sejam possíveis. Em algumas configurações, cada processador pode ter a sua própria memória principal e seus próprios canais de E/S além dos recursos compartilhados.

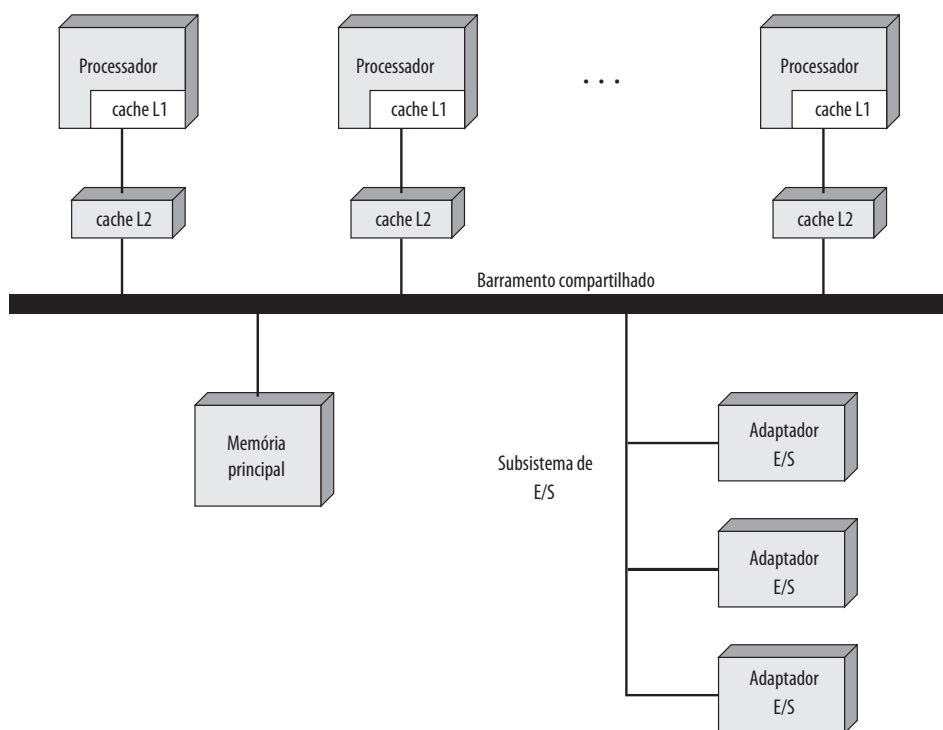
A organização mais comum para computadores pessoais, estações de trabalho e servidores é o barramento de tempo compartilhado. O barramento de tempo compartilhado é o mecanismo mais simples para construir um sistema multiprocessador (Figura 17.5). As estruturas e as interfaces são basicamente as mesmas para um sistema de um processador único que usa um barramento de interconexão. O barramento consiste de linhas de controle, endereço e dados. Para facilitar transferências DMA pelos processadores de E/S, os seguintes recursos são fornecidos:

- **Endereçamento:** deve ser possível distinguir os módulos no barramento para determinar a origem e o destino dos dados.
- **Arbitração:** qualquer módulo de E/S pode funcionar temporariamente como “mestre”. Um mecanismo é fornecido para arbitrar requisições concorrentes para o controle do barramento, usando algum tipo de esquema de prioridade.
- **Tempo compartilhado:** quando um módulo está controlando o barramento, outros módulos são bloqueados e devem, se necessário, suspender a operação até que o acesso ao barramento seja possível.

Estes recursos de uniprocessadores são utilizáveis diretamente em uma organização SMP. Neste último caso, existem agora múltiplos processadores, assim como múltiplos processadores de E/S, tentando obter o acesso a um ou mais módulos de memória pelo barramento.

A organização de barramento possui vários recursos atraentes:

- **Simplicidade:** esta é a abordagem mais simples para organização de multiprocessadores. A interface física e lógica de endereçamento, arbitração e tempo compartilhado de cada processador permanecem as mesmas, como em um sistema de um único processador.
- **Flexibilidade:** normalmente é fácil expandir o sistema anexando mais processadores ao barramento.
- **Confiabilidade:** o barramento é basicamente um meio passivo, e uma falha de qualquer dispositivo conectado não deve causar uma falha do sistema todo.

Figura 17.5 Organização de um multiprocessador simétrico

A principal desvantagem da organização de barramento é o desempenho. Todas as referências à memória passam pelo barramento comum. Assim, o tempo de ciclo do barramento limita a velocidade do sistema. Para melhorar o desempenho, é desejável equipar cada processador com uma memória cache. Isto deveria reduzir drasticamente o número de acessos ao barramento. Normalmente, as estações de trabalho e computadores pessoais SMP possuem dois níveis de cache, a cache L1 interno (o mesmo chip do processador) e cache L2 interno ou externo. Alguns processadores, hoje em dia, usam também uma cache L3.

O uso da cache introduz algumas novas considerações sobre projeto. Como cada cache local contém uma imagem de uma parte da memória, se uma palavra é alterada em uma cache, isso poderia, de uma maneira concebível, invalidar essa palavra em outra cache. Para prevenir isso, outros processadores devem ser avisados que ocorreu uma atualização. Este problema é conhecido como problema de *coerência de cache* e é normalmente resolvido pelo hardware, em vez de ser solucionado pelo sistema operacional. Discutimos esta questão na Seção 17.4.



Considerações sobre projeto dos sistemas operacionais para multiprocessadores

Um sistema operacional SMP gerencia processadores e outros recursos computacionais para que o usuário perceba um único sistema operacional controlando os recursos do sistema. Na verdade, tal configuração deveria aparecer como um sistema multiprogramado de um único processador. Tanto em SMP como em uniprocessadores, vários trabalhos ou processos podem estar ativos ao mesmo tempo e é responsabilidade do sistema operacional escalonar a sua execução e alocar recursos. O usuário pode construir aplicações que usam vários processos ou várias threads dentro do processo sem se preocupar se um processador único ou vários processadores estarão disponíveis. Assim, um sistema operacional para multiprocessadores deve fornecer toda a funcionalidade de um sistema multiprogramado mais os recursos adicionais para acomodar múltiplos processadores. Temos, dentre as principais questões de projeto:

- **Processos concorrentes simultâneos:** rotinas do SO precisam ser reentrantes para permitir que vários processadores executem o mesmo código do SO simultaneamente. Com múltiplos processadores execu-

tando mesmas ou diferentes partes do SO, tabelas do SO e estruturas de gerenciamento devem ser gerenciadas de acordo para evitar deadlock ou operações inválidas.

- **Escalonamento:** qualquer processador pode efetuar escalonamento, portanto os conflitos devem ser evitados. O escalonador deve atribuir processos prontos para processadores disponíveis.
- **Sincronização:** com múltiplos processos ativos tendo acesso potencial a espaços da memória compartilhada ou recursos de E/S compartilhados, cuidados devem ser tomados para fornecer sincronização eficiente. A sincronização é um recurso que reforça a exclusão mútua e ordenação de eventos.
- **Gerenciamento de memória:** gerenciamento de memória em um multiprocessador precisa lidar com todas as questões encontradas em máquinas de um processador, conforme discutido no Capítulo 8. Além disso, o sistema operacional precisa explorar o paralelismo disponível no hardware, tais como memórias com múltiplas portas para alcançar o melhor desempenho. Os mecanismos de paginação em diferentes processadores devem ser coordenados para reforçar a consistência quando vários processadores compartilham uma página ou um segmento para decidir sobre substituição de página.
- **Confiabilidade e tolerância a falhas:** o sistema operacional deve prover uma degradação sutil perante uma falha do processador. O escalonador e outras partes do sistema operacional devem reconhecer a perda de um processador e reestruturar as tabelas de gerenciamento de acordo.



Um mainframe SMP

A maioria dos PCs e estações de trabalho SMP usa uma estratégia de barramento de interconexão conforme ilustrado na Figura 17.5. É instrutivo analisar uma abordagem alternativa, a qual é usada para uma implementação recente da família de mainframes zSeries da IBM (SIEGEL, PFEFFER e MAGEE, 2004^b, MAK ET AL., 2004^c), chamada de z990. Esta família abrange uma faixa desde um uniprocessador com um cartão de memória até sistemas de alto nível com 48 processadores e 8 placas de memória. Os principais componentes da configuração são mostrados na Figura 17.6:

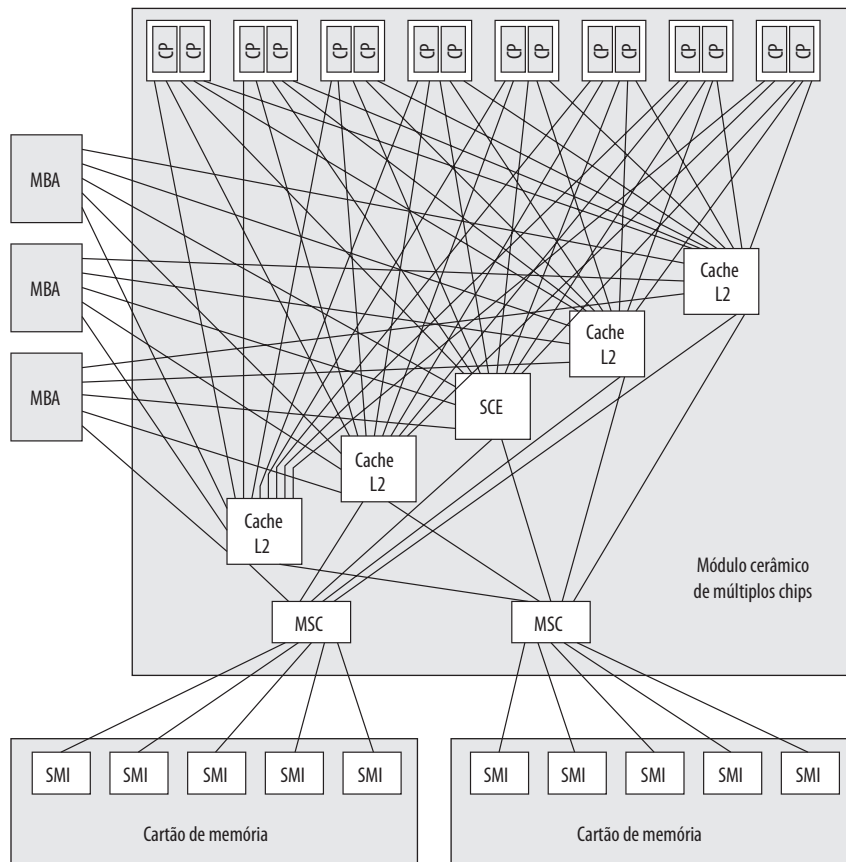
- **Chip de processador de dois núcleos (Dual-Core):** cada processador inclui dois processadores centrais idênticos (CP). O CP é um microprocessador CISC superescalar onde a maioria das instruções é executada por hardware e o restante é executado pelo microcódigo vertical. Cada CP inclui uma cache de instruções L1 de 256 KB e uma cache de dados L1 de 256 KB.
- **Cache L2:** cada cache L2 contém 32 MB. Caches L2 são arranjadas em grupos de cinco, com cada grupo suportando oito chips de processador e fornecendo acesso a todo o espaço da memória principal.
- **Elemento de controle do sistema (SCE, do inglês *system control element*):** o SCE faz arbitragem da comunicação do sistema e tem um papel central em manter a coerência de cache.
- **Controle do armazenamento principal (MSC, do inglês *main store control*):** o MSC interconecta caches L2 e a memória principal.
- **Cartão de memória:** cada cartão contém 32 GB de memória. O máximo configurável de memória consiste de 8 placas de memória para um total de 256 GB. Placas de memória se interconectam à MSC pelas interfaces de memória síncrona (SMI, do inglês *synchronous memory interfaces*).
- **Adaptador de barramento de memória (MBA, do inglês *memory bus adapter*):** o MBA provê uma interface para vários tipos de canais de E/S. Tráfego para/de canais vai diretamente para cache L2.

O microprocessador em z990 é relativamente incomum se comparado com outros processadores modernos porque, embora seja superescalar, ele executa instruções em ordem estritamente arquitetural. No entanto, ele compensa isso tendo um pipeline mais curto, e caches e TLB muito maiores, se comparado com outros processadores, além de outros recursos que melhoram a desempenho.

O sistema z990 engloba de um a quatro **livros**. Cada livro é uma unidade conectável contendo até 12 processadores com até 64 GB de memória, adaptadores de E/S e um elemento de controle de sistema (SCE) que conecta esses outros elementos. O SCE dentro de cada livro contém uma cache L2 de 32 MB que serve como um ponto central de coerência para esse livro específico. A cache L2 e a memória principal são acessíveis por um processador ou adaptador de E/S dentro desse livro ou qualquer outros dos três livros no sistema. Os chips SCE e cache L2 também se conectam com elementos correspondentes em outros livros em uma configuração de anel.

Existem vários recursos interessantes na configuração de SMP de z990, os quais discutimos agora:

- Interconexão chaveada.
- Caches L2 compartilhadas.

Figura 17.6 Estrutura do multiprocessador IBM z990

CP = processador central
 MBA = adaptador do barramento de memória
 MSC = controle do armazenamento principal
 SCE = elemento de controle do sistema
 SMI = interface de memória síncrona

INTERCONEXÃO CHAVEADA Um único barramento compartilhado é um arranjo comum em SMPs para PCs e estações de trabalho (Figura 17.5). Com este arranjo, o barramento único se torna um gargalo que afeta a escalabilidade (habilidade de expansão para tamanhos maiores) do projeto. O z990 lida com o problema de duas maneiras. Primeiro, a memória principal é dividida em múltiplos cartões, cada um com o seu controlador de armazenamento próprio que consegue lidar com acessos à memória em altas velocidades. A carga média de tráfego para memória principal é reduzida por causa dos caminhos independentes para partes separadas da memória. Cada livro inclui dois cartões de memória, para um total de oito cartões para a configuração máxima. Segundo, a conexão a partir dos processadores (mais precisamente a partir de caches L2) para um único cartão de memória não está na forma de um barramento compartilhado, e sim na forma de ligações ponto a ponto. Cada chip de processador possui uma ligação para cada uma das caches L2 no mesmo livro e cada cache L2 possui uma ligação, por meio de MSC, para cada um dos cartões de memória no mesmo livro.

Cada cache L2 se conecta apenas com os dois cartões de memória no mesmo livro. O controlador do sistema fornece ligações (não mostradas) para outros livros na configuração, de tal forma que toda a memória principal seja acessível para todos os processadores.

As ligações ponto a ponto em vez de um barramento fornecem conexões para os canais de E/S. Cada cache L2 em um livro se conecta com cada MBA desse livro. As MBAs, por sua vez, se conectam com os canais de E/S.

CACHES L2 COMPARTILHADAS Em um esquema típico de cache de dois níveis em um SMP, cada processador possui uma cache L1 dedicada e uma cache L2 também dedicada. Nos últimos anos, tem crescido o interesse pelo conceito de uma cache L2 compartilhada. Em uma versão anterior do seu mainframe SMP, conhecido como geração 3 (G3), a IBM fez uso de caches L2 dedicadas. Em suas versões posteriores (séries G4, G5 e série z900), uma cache L2 compartilhada foi usada. Duas considerações definiram esta mudança:

1. Ao mudar de G3 para G4, a IBM duplicou a velocidade dos microprocessadores. Se a organização G3 fosse mantida, um aumento significativo de tráfego no barramento teria ocorrido. Ao mesmo tempo, houve um desejo de reutilizar o máximo possível de componentes G3. Sem uma atualização significativa do barramento, o BSN teria se tornado um gargalo.
2. Análises de cargas de trabalho típicas de mainframe revelaram um alto grau de compartilhamento de instruções e dados entre os processadores.

Estas considerações levaram os projetistas da G4 a considerar o uso de uma ou mais caches L2, cada uma sendo compartilhada por vários processadores (cada processador tendo uma cache L1 dedicada no chip). À primeira vista, compartilhar uma cache L2 pode parecer uma ideia ruim. O acesso à memória a partir dos processadores deveria ser mais lento porque os processadores devem agora competir pelo acesso a uma única cache L2. No entanto, se uma quantidade suficiente de dados é, de fato, compartilhada por vários processadores, então uma cache compartilhada pode aumentar o rendimento ao invés de diminuí-lo. Dados que são compartilhados e encontrados na cache compartilhada são obtidos mais rapidamente do que se tivessem que ser obtidos pelos barramentos.



17.3 Coerência de cache e protocolo MESI

Nos atuais sistemas multiprocessadores, é comum haver um ou dois níveis de cache associados a cada processador. Esta organização é essencial para alcançar um desempenho razoável. No entanto, isso cria um problema conhecido como problema de **coerência de cache**. Em essência, o problema é: várias cópias dos mesmos dados podem existir em caches diferentes simultaneamente e, se for permitido aos processadores atualizarem as suas próprias cópias livremente, isso pode resultar em uma imagem da memória inconsistente. No Capítulo 4, definimos duas políticas de escrita comuns:

- **Write-back:** operações de escrita são feitas normalmente apenas na cache. A memória principal é atualizada apenas quando a linha de cache correspondente é retirada da cache.
- **Write-through:** todas as operações de escrita são feitas na memória principal e na cache, garantindo que a memória principal sempre esteja válida.

É claro que uma política de write-back pode resultar em inconsistência. Se duas caches contêm a mesma linha, e a linha é atualizada em uma cache, a outra cache terá um valor inválido sem saber. Leituras subsequentes dessa linha inválida produzem resultados inválidos. Mesmo com a política de write-through, inconsistências podem ocorrer a não ser que outras caches monitorem o tráfego de memória ou recebam alguma notificação direta sobre a atualização.

Nesta seção, analisamos brevemente várias abordagens para o problema de coerência de cache e depois focamos na abordagem que é a mais usada: protocolo MESI, do inglês *Modified, Exclusive, Shared, Invalid*. Uma versão deste protocolo é usada nas implementações de Pentium 4 e PowerPC.

Para qualquer protocolo de coerência de cache, o objetivo é deixar que variáveis locais recém-usadas cheguem à cache apropriada e permaneçam aí durante várias leituras e escritas, enquanto o protocolo é usado para manter a consistência das variáveis compartilhadas que podem estar em várias caches ao mesmo tempo. Abordagens para coerência de cache geralmente têm sido divididas em abordagens por hardware e por software. Algumas implementações adotam uma estratégia que envolve tanto elementos de software quanto de hardware. Mesmo assim, a classificação em abordagens por software e por hardware ainda é instrutiva e comumente usada ao analisar as estratégias de coerência de cache.



Soluções por software

Esquemas de coerência por cache por software tentam evitar a necessidade de hardware adicional, circuitos e lógicas, contando com compilador e sistema operacional para lidar com o problema. Abordagens de software são atraentes porque a sobrecarga de detectar problemas potenciais é transferida do tempo de execução para o tempo de compilação e a complexidade de projeto é transferida do hardware para o software. Por outro lado, abordagens

de software em tempo de compilação geralmente devem tomar decisões conservadoras, levando à utilização ineficiente da cache.

Os mecanismos de coerência baseados em compiladores efetuam uma análise do código para determinar que itens de dados podem se tornar problemas se armazenados na cache; eles ainda marcam esses itens de maneira adequada. O sistema operacional ou hardware, então, evita que esses itens indevidos sejam colocados em cache.

A abordagem mais simples é evitar que quaisquer variáveis de dados compartilhadas sejam colocadas na cache. Isto é conservador demais, porque uma estrutura de dados pode ser usada exclusivamente durante alguns períodos e pode ser efetivamente usada somente para leitura durante outros períodos. A coerência de cache se torna um problema apenas durante os períodos nos quais pelo menos um processo pode atualizar a variável e pelo menos um outro processado pode acessar a variável.

Abordagens mais eficientes analisam o código para determinar períodos seguros para variáveis compartilhadas. O compilador, então, insere instruções no código gerado para reforçar coerência de cache durante os períodos críticos. Uma série de técnicas tem sido desenvolvidas para efetuar a análise e para reforçar os resultados; veja análises em Lilja (1993^d) e Stenstrom (1990^e).



Soluções por hardware

Soluções baseadas em hardware são geralmente conhecidas como protocolos de coerência de cache. Estas soluções fornecem reconhecimento dinâmico em tempo de execução de condições de inconsistência potenciais. Como o problema é tratado apenas quando aparece de fato, há um uso mais eficiente de cache, o que leva a um desempenho melhor se comparado com a abordagem de software. Além disso, estas abordagens são transparentes ao programador e ao compilador, reduzindo o trabalho no desenvolvimento de software.

Esquemas de hardware diferem em uma série de particularidades, incluindo onde a informação sobre estado das linhas por dados é guardada, como essa informação é organizada, onde a coerência é reforçada e os mecanismos de reforço. Em geral, os esquemas por hardware podem ser divididos em duas categorias: protocolos de diretório e protocolos de detecção.

PROTOCOLOS DE DIRETÓRIO Protocolos de diretório coletam e mantêm a informação sobre onde as cópias das linhas residem. Normalmente, há um controlador centralizado que é parte do controlador da memória principal e um diretório que é guardado na memória principal. O diretório contém informação de estado global sobre o conteúdo de várias caches locais. Quando um controlador de cache individual faz uma requisição, o controlador centralizado verifica e emite comandos necessários para transferência de dados entre memória e caches e entre caches. Ele é responsável também por guardar a informação de estado atualizada; portanto, cada ação local que pode afetar o estado global de uma linha deve ser reportada para o controlador central.

Normalmente, o controlador mantém a informação sobre quais processadores têm uma cópia de quais linhas. Antes que um processador possa escrever em uma cópia local de uma linha, ele deve requisitar o acesso exclusivo para a linha ao controlador. Antes de conceder esse acesso exclusivo, o controlador envia uma mensagem para todos os processadores com uma cópia da cache dessa linha, forçando cada processador a invalidar a sua cópia. Depois de receber o reconhecimento de volta de cada processador, o controlador concede acesso exclusivo para o processador requisitante. Quando outro processador tenta ler uma linha que está exclusivamente concedida para outro processador, ele envia uma notificação de falha para o controlador. O controlador, então, emite um comando para o processador que guarda essa linha para que o processador escreva-a de volta na memória principal. A linha agora pode ser compartilhada para leitura pelo processador original e processador requisitante.

Esquemas de diretório tem a desvantagem de um gargalo central e de uma sobrecarga de comunicação entre os vários controladores de cache e o controlador central. No entanto, eles são eficientes em sistemas de grande escala que envolvem vários barramentos ou algum outro esquema complexo de interconexão.

PROTOCOLOS DE MONITORAÇÃO (Snoopy Protocols) Protocolos *snoopy* distribuem a responsabilidade de manter a coerência de cache entre todos os controladores de cache em um multiprocessador. Uma cache deve reconhecer quando uma linha que ela guarda é compartilhada com outras caches. Quando uma ação de atualização é feita em uma linha compartilhada na cache, ela deve ser anunciada para todas as outras caches por meio de um mecanismo de difusão (*broadcast*). Cada controlador de cache é capaz de “monitorar” na rede essas notificações de dispersão e reagir de acordo.

Protocolos de *snoopy* encaixam-se perfeitamente em um multiprocessador baseado em barramento, porque o barramento compartilhado fornece um meio simples para difusão e monitoramento. No entanto, como um dos

objetivos do uso de caches locais é evitar acessos ao barramento, cuidado deve ser tomado para que o tráfego de barramento aumentado para difusão e monitoramento não anule os ganhos do uso de caches locais.

Duas abordagens básicas para protocolo de detecção foram exploradas: write invalidate e write update (ou write broadcast). Com um protocolo de write invalidate, pode haver vários leitores, mas apenas um escritor ao mesmo tempo. Inicialmente, uma linha pode ser compartilhada entre várias caches para propósitos de leitura. Quando uma das caches deseja escrever na linha, ela primeiramente emite um aviso que invalida essa linha em outras caches, tornando a linha exclusiva para a cache que estará escrevendo. Uma vez a linha se tornando exclusiva, o processador proprietário pode fazer as escritas locais e baratas até que algum outro processador solicite a mesma linha.

Em um protocolo de write updates, pode haver vários escritores como também vários leitores. Quando um processador deseja atualizar uma linha compartilhada, a palavra a ser atualizada é distribuída para todas as outras e as caches que contêm essa linha podem atualizá-la.

Nenhum destes dois protocolos é superior a outro em todas as situações. O desempenho depende do número de caches locais e do padrão de leituras e escritas de memória. Alguns sistemas implementam protocolos adaptáveis que implementam ambos os mecanismos, write invalidate e write update.

A abordagem write invalidate é a mais usada em sistemas multiprocessadores comerciais, como Pentium 4 e PowerPC. Ela marca o estado de cada linha de cache (usando dois bits extras na marcação da cache) como modificada, exclusiva, compartilhada ou inválida. Por esta razão, o protocolo write invalidate é chamado de MESI.¹ No restante desta seção, analisamos o seu uso entre caches locais por meio de um multiprocessador. Para simplicidade da apresentação, não analisamos os mecanismos envolvidos em coordenação entre os níveis 1 e 2 localmente, assim como o tempo de coordenação pelo multiprocessador distribuído. Isso não adicionaria nenhum princípio novo, porém complicaria muito a discussão.



O protocolo MESI

Para fornecer a consistência de cache em um SMP, a cache de dados frequentemente suporta um protocolo conhecido como MESI. Para o MESI, a cache de dados inclui dois bits de estado para cada *tag*, para que cada linha possa estar em um dos quatro estados:

- **Modificada:** a linha na cache foi modificada (diferente da memória principal) e está disponível apenas nesta cache.
- **Exclusiva:** a linha na cache é a mesma da memória principal e não está presente em nenhuma outra cache.
- **Compartilhada:** a linha na cache é a mesma da memória principal e pode estar presente em outra cache.
- **Inválida:** a linha na cache não contém dados válidos.

A Tabela 17.1 resume o significado dos quatro estados, e a Figura 17.7 mostra um diagrama de estado para o protocolo MESI. Tenha em mente que cada linha de cache tem os seus próprios bits de estado e, portanto, a sua própria instância do diagrama de estado. A Figura 17.7a mostra as transições que ocorrem por causa das ações iniciadas pelo processador associado a essa cache. A Figura 17.7b mostra as transições que ocorrem por causa dos eventos que são detectados no barramento comum. Esta apresentação de diagramas de estado separados para ações de iniciar processador e iniciar bar-

Tabela 17.1 Estado das linhas da cache MESI

	M Modificada	E Exclusiva	S (shared) Compartilhada	I Inválida
Esta linha da cache está válida?	Sim	Sim	Sim	Não
A cópia da memória está...	desatualizada	válida	válida	—
Há cópias em outras caches?	Não	Não	Talvez	Talvez
Uma escrita nesta linha...	não vai para barramento	não vai para barramento	vai para barramento e atualiza a cache	vai diretamente para barramento

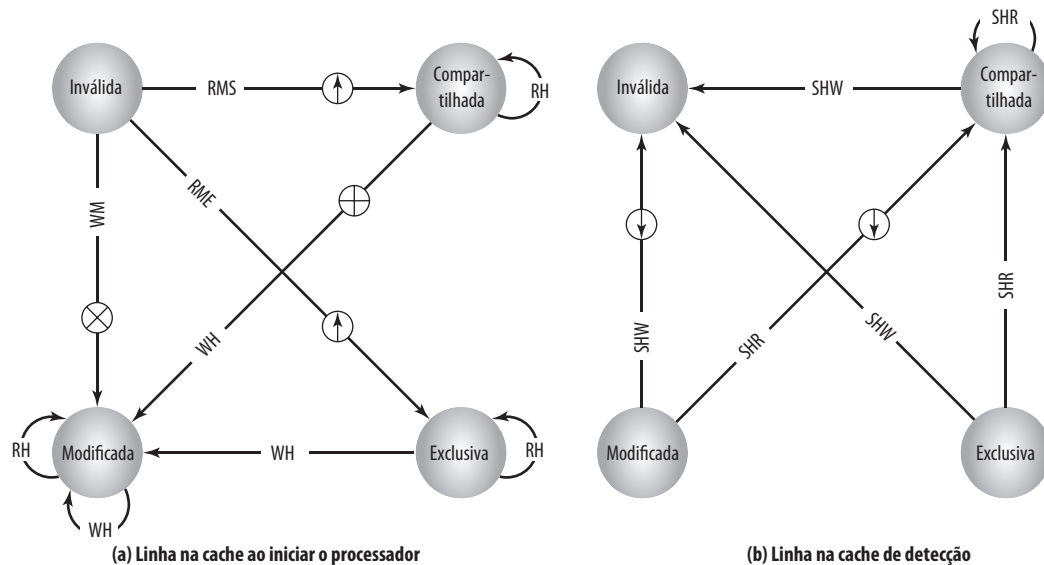
¹ Nota do tradutor: a letra S na sigla MESI vem do inglês *Shared* (compartilhada).

ramento ajuda a esclarecer a lógica do protocolo MESI. A qualquer momento, a linha da cache está em um estado único. Se o próximo evento vem do processador anexo, então a transição é ditada pela Figura 17.7a, e se o próximo evento vem do barramento, a transição é ditada pela Figura 17.7b. Vamos analisar essas transições em mais detalhes.

LEITURA COM FALHA (READ MISS) Quando ocorre uma falha de leitura em uma cache local, o processador inicia uma leitura de memória para ler a linha da memória principal que contém o endereço que está faltando. O processador insere um sinal no barramento que avisa todos os outros processadores/unidades de cache para detectarem a transação. Há vários desfechos possíveis:

- Se outra cache possui uma cópia limpa (não modificada desde a leitura da memória) da linha no estado exclusivo, ela retorna um sinal indicando que compartilhe essa linha. O processador que respondeu passa o estado da sua cópia de exclusiva para compartilhada e o processador que iniciou lê a linha da memória principal e passa a linha na sua cache de inválida para compartilhada.
- Se uma ou mais caches têm uma cópia limpa da linha no estado compartilhado, cada uma delas sinaliza que compartilha essa linha. O processador que iniciou lê a linha e passa a linha na sua cache de inválida para compartilhada.
- Se outra cache tem uma cópia modificada da linha, então essa cache bloqueia a leitura de memória e fornece a linha para a cache que requisitou por meio do barramento compartilhado. A cache que respondeu muda, então, a sua linha de modificada para compartilhada.² A linha enviada para a cache requisitante é também recebida e processada pelo controlador de memória, que guarda o bloco na memória.

Figura 17.7 Diagrama de transição do estado do protocolo MESI



RH = leitura com acerto (hit)	⬇️ Cópia de linha suja
RMS = leitura com falha, compartilhada	⊕ Transação inválida
RME = leitura com falha, exclusiva	⊗ Leitura com intenção de modificar
WH = escrita com acerto (hit)	⬆️ Preenchimento de linha da cache
WM = escrita com falha	
SHR = detectar acerto na leitura	
SHW = detectar acerto na escrita ou leitura com intenção de modificar	

² Em algumas implementações, a cache com a linha modificada sinaliza o processador que iniciou para tentar novamente. Enquanto isso, o processador com a cópia modificada segura o barramento, escreve a linha modificada de volta na memória principal e passa a linha na sua cache de modificada para compartilhada. Subsequentemente, o processador requisitante tenta novamente e descobre que um ou mais processadores possuem uma cópia limpa da linha no estado compartilhado, conforme descrito no ponto anterior.

- Se nenhuma outra cache tem uma cópia da linha (limpa ou modificada), então nenhum sinal é retornado. O processador que iniciou lê a linha e passa a linha na sua cache de inválida para exclusiva.

LEITURA COM ACERTO (READ HIT) Quando uma leitura com acerto ocorre em uma linha que está atualmente na cache local, o processador simplesmente lê o item requerido. Não há mudança de estado: o estado permanece modificado, compartilhado ou exclusivo.

ESCRITA COM FALHA Quando ocorre uma escrita com falha na cache local, o processador inicia uma leitura de memória para ler a linha da memória principal contendo o endereço que faltou. Para este propósito, o processador emite um sinal no barramento que significa *leitura com intenção de modificar* (RWITM, do inglês *read-with-intent-to-modify*). Quando a linha é carregada, ela é imediatamente marcada como modificada. Em relação a outras caches, dois cenários possíveis antecedem o carregamento da linha de dados.

Primeiro, alguma outra cache pode ter uma cópia modificada dessa linha (estado = modificado). Neste caso, o processador alertado sinaliza ao processador iniciante que outro processador tem uma cópia modificada da linha. O processador que iniciou entrega o barramento e espera. O outro processador obtém acesso ao barramento, escreve a linha de cache modificada de volta na memória principal e passa o estado da linha de cache para inválida (porque o processador que iniciou vai modificar esta linha). Subsequentemente, o processador que iniciou emite novamente um sinal RWITM para o barramento e depois lê a linha da memória principal, modifica a linha na cache e muda a linha para estado modificado.

O segundo cenário é quando nenhuma outra cache possui uma cópia modificada da linha requisitada. Neste caso, nenhum sinal é retornado e o processador que iniciou continua a ler a linha e a modificá-la. Enquanto isso, se uma ou mais caches possuem uma cópia limpa da linha no estado compartilhado, cada cache invalida a sua cópia da linha e se uma cache tiver uma cópia limpa da linha no estado exclusivo, ela invalida a sua cópia da linha.

ESCRITA COM ACERTO (WRITE HIT) Quando ocorre uma escrita com sucesso em uma linha que está atualmente na cache local, o efeito depende do estado atual dessa linha na cache local:

- **Compartilhada:** antes de efetuar atualização, o processador deve obter a propriedade exclusiva da linha. O processador sinaliza a sua intenção no barramento. Todo processador que tem uma cópia compartilhada da linha na sua cache passa-a de compartilhada para inválida. O processador que iniciou então efetua a atualização e passa a sua cópia da linha de compartilhada para modificada.
- **Exclusiva:** o processador já possui o controle exclusivo desta linha, então ele simplesmente efetua a atualização e passa a sua cópia da linha de exclusiva para modificada.
- **Modificada:** o processador já possui o controle exclusivo desta linha e a linha está marcada como modificada, então ele simplesmente efetua a atualização.

CONSISTÊNCIA DE CACHE L1-L2 Até agora descrevemos protocolos de coerência de cache em termos de atividade cooperativa entre caches conectadas ao mesmo barramento ou outro recurso de interconexão de SMP. Normalmente, estas caches são caches L2 e cada processador possui também uma cache L1 que não se conecta diretamente ao barramento e, portanto, não pode fazer parte de um protocolo de detecção. Assim, algum esquema é necessário para manter a integridade de dados entre ambos os níveis de cache e entre todas as caches na configuração SMP.

A estratégia é estender o protocolo MESI (ou qualquer protocolo de coerência de cache) para caches L1. Assim, cada linha na cache L1 inclui bits para indicar o estado. Basicamente, o objetivo é o seguinte: para cada linha que está presente na cache L2 e na sua cache L1 correspondente, o estado da linha L1 deve seguir o estado da linha L2. Uma forma simples de fazer isso é adotar a política de write through na cache L1; neste caso, a escrita direta é para a cache L2 e não para a memória. A política de write through de L1 força qualquer modificação em uma linha L1 para a cache L2 e assim a torna visível para outras caches L2. O uso da política de write through de L1 requer que o conteúdo de L1 seja um subconjunto do conteúdo L2. Isso, por sua vez, sugere que a associatividade da cache L2 seja igual ou maior que a associatividade de L1. A política de *write-through* de L1 é usada no IBM S/390 SMP.

Se a cache L1 tem uma política write-back, a relação entre as duas caches é mais complexa. Existem várias abordagens para manter a coerência. Por exemplo, a abordagem usada no Pentium II é descrita em detalhes em Shanley (2005').



17.4 Multithreading e chips multiprocessadores

A medida mais importante de desempenho para um processador é a taxa em que ele executa as instruções. Isso pode ser expresso como:

$$\text{Taxa MIPS} = f \times \text{IPC}$$

onde f é a frequência de clock do processador, em MHz, e IPC (instruções por ciclo) é o número médio de instruções executadas por ciclo. De acordo com isso, os projetistas têm perseguido o objetivo de aumentar o desempenho em duas frentes: aumento de frequência de clock e aumento de número de instruções executadas ou, mais apropriadamente, o número de instruções completadas durante um ciclo do processador. Conforme vimos em capítulos anteriores, os projetistas aumentaram o IPC usando um pipeline de instruções e pipelines múltiplos paralelos de instruções em uma arquitetura superescalar. Com projetos de pipeline e pipelines múltiplos, o principal problema é maximizar a utilização de cada estágio do pipeline. Para melhorar o rendimento, os projetistas criaram mecanismos cada vez mais complexos, como executar algumas instruções em uma ordem diferente da forma que ocorrem no fluxo de instruções e começar a execução de instruções que podem nunca ser necessárias. Mas como foi discutido na Seção 2.2, esta abordagem pode estar alcançando o limite por causa da complexidade e dos problemas de consumo de energia.

Uma abordagem alternativa, a qual permite um grau mais alto de paralelismo em nível de instruções sem aumentar a complexidade dos circuitos ou consumo de energia, é chamada de *multithreading*. Basicamente, o fluxo de instruções é dividido em vários fluxos menores, conhecidos como *threads*, de modo que cada *thread* possa ser executada em paralelo.

A variedade de projetos específicos de *multithreading* realizada nos sistemas comerciais e nos experimentais é muito grande. Nesta seção, fazemos uma breve análise dos principais conceitos.



Multithreading implícito e explícito

O conceito de *thread* usado na discussão sobre processadores *multithread* pode ou não ser o mesmo que o conceito de *threads* de software em sistemas operacionais multiprogramados. Será útil definir os termos rapidamente:

- **Processo:** uma instância de um programa executando em um computador. Um processo engloba duas características principais:
 - **Posse do recurso:** um processo inclui um espaço de endereço virtual para guardar a imagem do processo; a imagem do processo é coleção de programa, dados, pilhas e atributos que definem o processo. De tempos em tempos, a um processador pode ser dada a posse (ou controle) de recursos, como memória principal, canais de E/S, dispositivos de E/S e arquivos.
 - **Escalonamento/execução:** a execução de um processo segue um caminho de execução (rastro) por um ou mais programas. Esta execução pode ser intercalada com a de outros processos. Assim, um processo possui um estado de execução (Executando, Pronto etc.) e uma prioridade de despacho, e é a entidade que é escalonada e despachada pelo sistema operacional.
- **Troca de processos:** uma operação que troca em um processador de um processo para outro, salvando todos os dados de controle do processador, registradores e outras informações do primeiro e substituindo-as com informações de processo do segundo.³
- **Thread:** uma unidade de trabalho dentro de um processo que pode ser despachada. Ela inclui um contexto de processador (o qual inclui o contador de programa e o ponteiro de pilha) e sua própria área de dados para uma pilha (para possibilitar desvio de subrotinas). Uma *thread* executa sequencialmente e pode ser interrompida para que o processador possa se dedicar a outra *thread*.
- **Troca de thread:** o ato de trocar o controle do processador de uma *thread* para outra dentro do mesmo processo. Normalmente, este tipo de troca é muito menos custoso do que uma troca de processo.

³ O termo *troca de contexto* é frequentemente encontrado em literatura e livros sobre SO. Infelizmente, embora a maior parte da literatura use este termo para se referir ao que é chamado aqui de troca de processo, outras fontes o usam para se referir à troca de *thread*. Para evitar ambiguidade, o termo não é usado neste livro.

Desta forma, uma *thread* preocupa-se com escalonamento e execução, enquanto um processo se preocupa com escalonamento/execução e posse de recursos. Várias *threads* dentro de um processo compartilham os mesmos recursos. É por isso que uma troca de *thread* consome bem menos tempo do que uma troca de processo. Os sistemas operacionais tradicionais, como versões anteriores do Unix, não suportavam *threads*. A maioria de sistemas operacionais modernos, como Linux, outras versões de Unix e Windows, suporta *threads*. Uma distinção é feita entre *threads* em nível de usuário, as quais são visíveis para o programa da aplicação, e *threads* em nível de kernel, as quais são visíveis apenas para o sistema operacional. Ambas podem ser referidas como *threads* explícitas, definidas em software.

Todos os processadores comerciais e a maioria de processadores experimentais até hoje têm usado *multithreading* explícito. Esses sistemas executam instruções de diferentes *threads* explícitas diferentes de forma concorrente, ou com intercalação de instruções de diferentes *threads* em pipelines compartilhados ou com execução paralela em pipelines paralelos. *Multithreading* implícito refere-se à execução concorrente de múltiplas *threads* extraídas de um único programa sequencial. Estas *threads* implícitas podem ser definidas estaticamente pelo compilador ou dinamicamente pelo hardware. No restante desta seção, consideramos *multithreading* explícito.



Abordagens para *multithreading* explícito

Um processador *multithread* deve prover no mínimo um contador de programa separado para cada *thread* de execução a ser executada concorrentemente. Os projetos diferem em quantidade e tipo de hardware adicional usado para suportar execução de *threads* concorrentes. Em geral, a busca de instruções ocorre na base de *threads*. O processador trata cada *thread* separadamente e pode usar uma série de técnicas para otimizar a execução de uma *thread*, incluindo previsão de desvio, renomeação de registradores e técnicas superescalares. Desta forma, alcança-se paralelismo em nível de *threads*, o que pode prover melhor desempenho quando casado com paralelismo em nível de instruções.

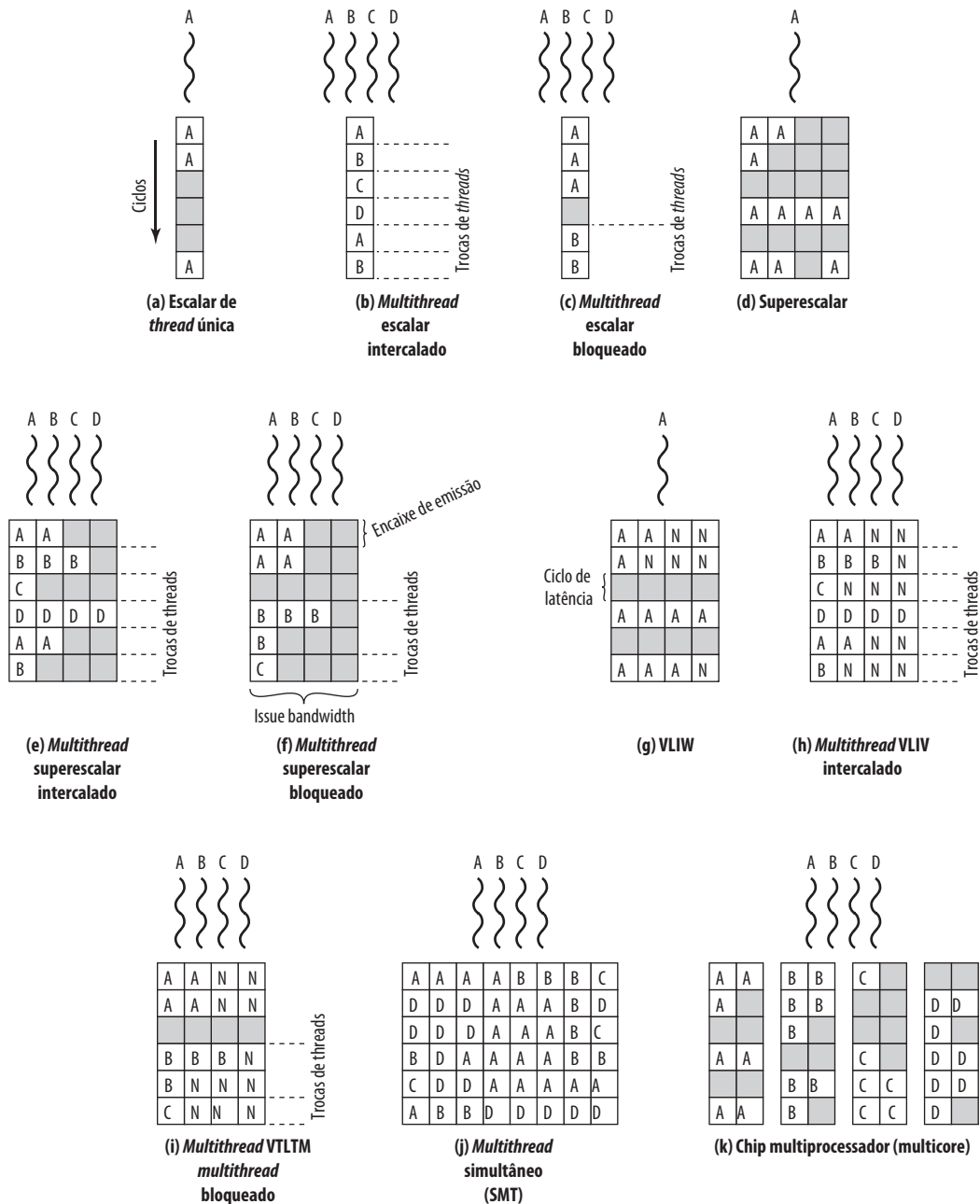
Em termos gerais, existem quatro abordagens principais para *multithreading*:

- ***Multithreading* intercalado:** isto é conhecido também como ***multithreading* de granularidade fina**. O processador lida com dois ou mais contextos de *thread* ao mesmo tempo, trocando de uma *thread* para outra a cada ciclo de clock. Se uma *thread* é bloqueada por causa das dependências de dados ou latências de memória, ela é pulada e uma *thread* pronta é executada.
- ***Multithreading* bloqueado:** isto é conhecido também como ***multithreading* de granularidade grossa**. As instruções de uma *thread* são executadas sucessivamente até que ocorra um evento que possa causar atraso, como uma falha de cache. Este evento induz uma troca para outra *thread*. Esta abordagem é eficiente em um processador em-ordem que iria parar o pipeline num evento de atraso como uma falha de cache.
- ***Multithreading* simultâneo (SMT):** instruções são enviadas simultaneamente a partir de múltiplas *threads* para unidades de execução de um processador superescalar. Isto combina a capacidade de envio de instruções superescalares com o uso de múltiplos contextos de *threads*.
- **Chip multiprocessadores:** neste caso, o processador inteiro é replicado em um único chip e cada processador lida com *threads* separadas. A vantagem desta abordagem é que a área de lógica disponível em um chip é usada eficientemente sem depender da sempre crescente complexidade no projeto do pipeline. Isto é conhecido como multicore; analisamos este tópico separadamente no Capítulo 18.

Para as duas primeiras abordagens, instruções de diferentes *threads* não são executadas simultaneamente. Em vez disso, o processador é capaz de trocar rapidamente de uma *thread* para outra, usando um conjunto de registradores diferente e outra informação de contexto. Isso resulta em uma utilização melhor dos recursos de execução do processador e evita uma penalidade grande por causa das falhas de cache e outros eventos de atraso. A abordagem SMT envolve a verdadeira execução simultânea de instruções de diferentes *threads*, usando recursos de execução replicados. Chips multiprocessadores possibilitam também execução simultânea de instruções de diferentes *threads*.

A Figura 17.8, baseada em uma figura de Ungerer, Rubic e Silc (2002⁹), ilustra algumas arquiteturas possíveis de pipeline, que envolvem *multithreading*, e as compara com as abordagens que não usam *multithreading*. Cada linha horizontal representa um slot (ou slots) de envio em potencial para um ciclo de execução único; ou seja, a largura de cada linha corresponde ao número máximo de instruções que podem ser emitidas em um único ciclo de clock.⁴ A dimensão vertical representa a sequência de tempo de ciclos de clock.

4 Slots de envio são as posições das quais as instruções podem ser enviadas em um dado ciclo de clock. Lembre que, no Capítulo 14, vimos que o envio de instrução é o processo de inicializar a execução da instrução em unidades funcionais do processador. Isto ocorre quando uma instrução se move do estágio de decodificação no pipeline para o primeiro estágio de execução no pipeline.

Figura 17.8 Abordagens para execução de múltiplos *threads*

Um slot vazio (sombreado) representa um slot de execução não usado em um pipeline. Um no-op (*no operation*) é indicado por um N.

As três primeiras ilustrações na Figura 17.8 mostram abordagens diferentes com um processador escalar (isto é, emissão única):

- **Thread escalar único:** este é o pipeline simples encontrado em máquinas RISC e CISC tradicionais, sem *multithreading*.
- **Multithread escalar intercalado:** esta é a abordagem de *multithreading* mais fácil de ser implementada. Ao trocar de uma *thread* para outra em cada ciclo de clock, os estágios do pipeline podem ser mantidos

totalmente ocupados, ou quase totalmente ocupados. O hardware deve ser capaz de trocar de um contexto de *thread* para outro entre os ciclos.

- **Multithread escalar bloqueado:** neste caso, uma única *thread* é executada até que ocorra um evento de atraso que pararia o pipeline, momento em que o processador troca para outra *thread*.

A Figura 17.8c mostra uma situação na qual o tempo para executar uma troca de *thread* é de um ciclo, enquanto a 17.8b mostra que a troca de *thread* ocorre em zero ciclos. No caso de *multithread* intercalado, assume-se que não há dependências de dados ou controle entre *threads*, o que simplifica o projeto do pipeline e deveria, portanto, permitir a troca de *thread* sem nenhum atraso. No entanto, dependendo do projeto e da implementação específica, *multithread* de bloqueio pode requerer um ciclo de clock para efetuar a troca de *thread*, conforme ilustrado na Figura 17.8. Isso é verdade se a instrução obtida dispara a troca de *thread* e deve ser descartada do pipeline (UNGERER, RUBIC E SILC 2003^h).

Embora a *multithread* intercalado pareça oferecer melhor utilização do processador do que a *multithread* de bloqueio, ele consegue isso sacrificando o desempenho de *thread* únicos. Vários *threads* competem pelos recursos de cache, o que eleva a probabilidade de uma falha de cache para uma determinada *thread*.

Mais oportunidades para execução paralela estão disponíveis se o processador puder enviar várias instruções por ciclo. As figuras de 17.8d a 17.8i ilustram um número de variações entre processadores que possuem hardware para enviar quatro instruções por ciclo. Em todos estes casos, apenas as instruções de uma única *thread* são emitidas em um único ciclo. As seguintes alternativas são ilustradas:

- **Superescalar:** esta é a abordagem superescalar básica sem nenhum *multithread*. Até há relativamente pouco tempo, esta era a abordagem mais poderosa para permitir paralelismo dentro de um processador. Observe que, durante alguns ciclos, nem todos os slots de envio são usados. Durante esses ciclos, menos que o número máximo de instruções é usado; chamamos isso de *perda horizontal*. Durante outros ciclos de instrução, nenhum slot de envio é usado; estes são os ciclos quando nenhuma instrução pode ser enviada; chamamos isso de *perda vertical*.
- **Multithread superescalar intercalado:** durante cada ciclo são emitidas tantas instruções quantas forem possíveis a partir de um único *thread*. Com esta técnica, atrasos potenciais por causa das trocas de *threads* são eliminados, conforme discutido anteriormente. No entanto, o número de instruções enviado em qualquer ciclo ainda é limitado pelas dependências que existem dentro de qualquer *thread*.
- **Multithread superescalar bloqueado:** novamente, as instruções de apenas uma *thread* podem ser emitidas durante qualquer ciclo e a *multithread* bloqueado é usado.
- **Slot de envio (VLIW, do inglês very long instruction word):** uma arquitetura VLIW, como IA-64, coloca várias instruções em uma única palavra. Normalmente, uma VLIW é construída pelo compilador, o qual coloca operações que podem ser executadas em paralelo na mesma palavra. Em uma máquina VLIW simples (Figura 17.8g), se não for possível preencher a palavra completamente com instruções a serem emitidas em paralelo, no-ops são usados.
- **VLIW Multithread intercalado:** esta abordagem deveria fornecer eficácia semelhante àquela provida por *multithreading* intercalada em uma arquitetura superescalar.
- **Multithread VLIW bloqueado:** esta abordagem deveria fornecer eficácia semelhante àquela provida por *multithread* bloqueado em uma arquitetura superescalar.

Duas últimas abordagens ilustradas na Figura 17.8 possibilitam execução paralela e simultânea de várias *threads*:

- **Multithreading simultâneo:** a Figura 17.8i mostra um sistema capaz de emitir 8 instruções ao mesmo tempo. Se um *thread* possui um alto grau de paralelismo em nível de instruções, ela pode, em alguns ciclos, ser capaz de preencher todos os slots horizontais. Em outros ciclos, as instruções de duas ou mais *threads* podem ser enviados. Se *threads* suficientes estão ativos, normalmente seria possível enviar o número máximo de instruções em cada ciclo, fornecendo um nível alto de eficiência.
- **Chip multiprocessador (multicore):** a Figura 17.8k mostra um chip que contém quatro processadores, cada um tendo um processador superescalar de envio dupla. A cada processador é atribuído um *thread* a partir do qual ele pode enviar até duas instruções por ciclo. Discutiremos computadores multicore no Capítulo 18.

Comparando as figuras 17.8j e 17.8k, vemos que um chip multicore com a mesma capacidade de enviados de instruções como um SMT não pode alcançar o mesmo grau de paralelismo em nível de instruções. Isso ocorre porque o chip multicore não é capaz de esconder os atrasos enviando instruções de outros *threads*. Por outro lado,

o chip multicore deve ter um desempenho melhor que um processador superescalar com a mesma capacidade de envio de instruções porque as perdas horizontais serão maiores para o processador superescalar. Além disso, é possível usar *multithread* dentro de cada processador em um chip multicore, e isso é feito em algumas máquinas atuais.



Exemplo de sistemas

PENTIUM 4 Modelos mais recentes de Pentium 4 usam uma técnica de *multithread* à qual a literatura da Intel refere-se como *hyperthreading* (MARR et al. 2002¹). Basicamente, a abordagem do Pentium 4 é usar SMT com suporte para dois *threads*. Assim, um único processador *multithread* torna-se logicamente dois processadores.

IBM POWER5 O chip IBM Power5, o qual é usado em produtos PowerPC de alto nível, combina o chip multiprocessador com SMT (KALLA, SINHAROY e TENDLER, 2004²). O chip possui dois processadores separados, sendo que cada um é um processador *multithread* capaz de suportar dois *threads* concorrentemente usando SMT. É interessante que os projetistas simularam várias alternativas e descobriram que dois processadores SMT two-way em um único chip fornecem desempenho superior do que um processador SMT único four-ways. As simulações mostraram que *multithread* adicional, além do suporte para dois *threads*, pode diminuir o desempenho por causa do trabalho com a cache, os dados de uma *thread* deslocam os dados necessários para outra *thread*.

A Figura 17.9 mostra o diagrama de fluxo da instrução de IBM Power5. Apenas poucos elementos no processador precisam ser replicados, com elementos separados dedicados a *threads* separadas. Dois contadores de programa são usados. O processador alterna a leitura de instruções, até oito por vez, entre dois *threads*. Todas as instruções são armazenadas em uma cache comum de instruções e compartilham um recurso de tradução de instruções que fazem a decodificação parcial da instrução. Quando um desvio condicional é encontrado, o recurso de previsão de desvio prevê a direção do desvio e, se possível, calcula o endereço alvo. Para prever o alvo do retorno de uma sub-rotina, o processador usa uma pilha de retorno, uma para cada *thread*.

Instruções então movem-se para dois buffers de instruções separados. Depois, com base na prioridade de *threads*, um grupo de instruções é selecionado e decodificado em paralelo. A seguir, as instruções fluem por um recurso de renomeação de registradores na ordem do programa. Os registradores lógicos são mapeados para registradores físicos. O Power5 possui 120 registradores físicos de uso geral e 120 registradores físicos de ponto flutuante. As instruções então são movidas para filas de envio. A partir das filas de envio, as instruções são emitidas usando *multithread* simétrico. Isto é, o processador tem uma arquitetura superescalar e pode emitir instruções a partir de uma ou ambos os *threads* em paralelo. No fim do pipeline, os recursos de *threads* separados são necessários para encerrar a instrução.



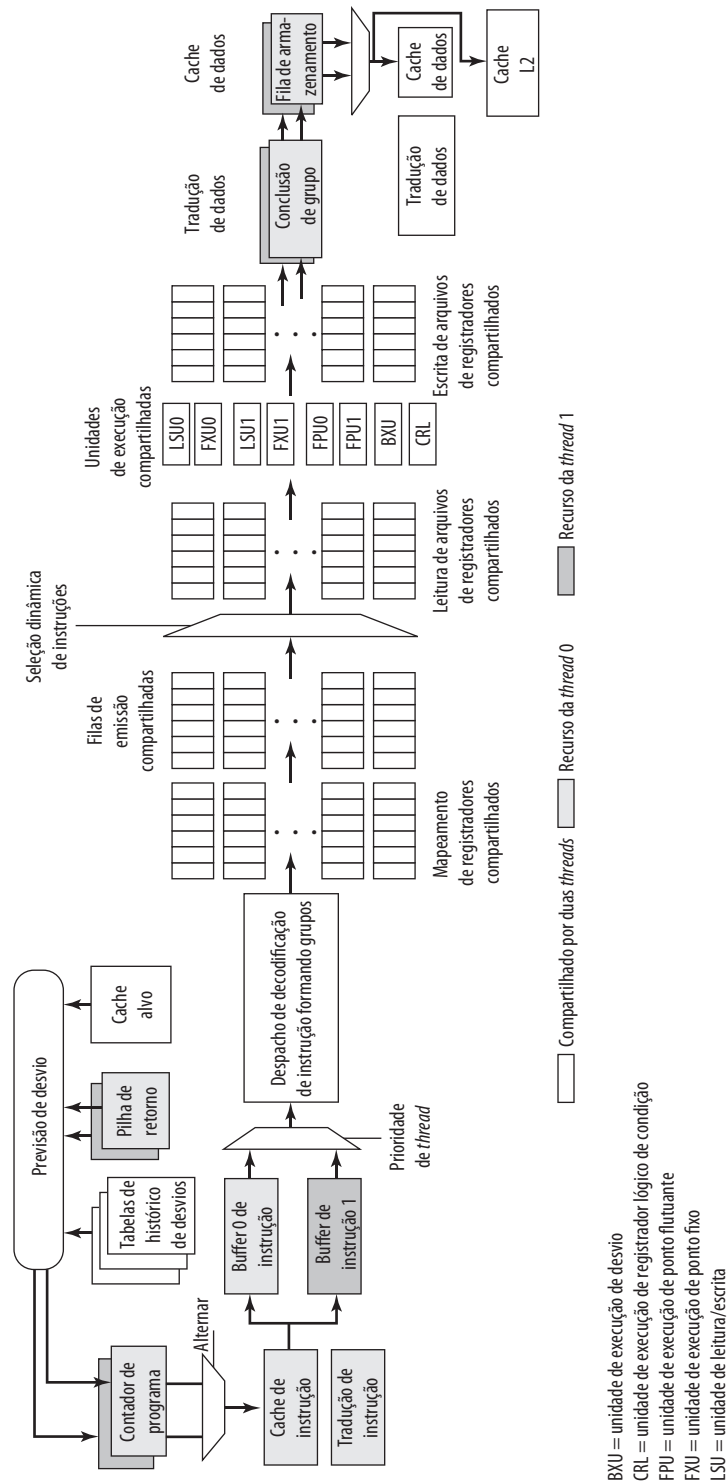
17.5 Clusters

Um recurso importante e relativamente recente no projeto de computadores é o *clustering*. *Clusters* são uma alternativa para multiprocessamento simétrico como uma abordagem para fornecer alto desempenho e disponibilidade e são bastante atraentes para aplicações de servidores. Podemos definir um *cluster* como um grupo de computadores completos interconectados trabalhando juntos, como um recurso computacional unificado que pode criar a ilusão de ser uma única máquina. O termo *computador completo* significa um sistema que pode funcionar por si só, à parte do *cluster*; na literatura, cada computador em um *cluster* normalmente é chamado de um *nó*.

Brewer (1997³) lista quatro benefícios que podem ser conseguidos com *cluster*. Estes podem ser pensados também como objetivos ou requisitos de projeto:

- **Escalabilidade absoluta:** é possível criar *clusters* grandes que ultrapassam em muito o poder de máquinas maiores máquinas que trabalham sozinhos. Um *cluster* pode ter dezenas, centenas ou até milhares de máquinas, cada uma sendo um multiprocessador.
- **Escalabilidade incremental:** um *cluster* é configurado de tal forma que é possível adicionar novos sistemas ao *cluster* em incrementos pequenos. Assim, um usuário pode começar com um sistema modesto e expandi-lo conforme a necessidade, sem ter que fazer uma atualização grande onde um sistema existente pequeno é substituído por um sistema maior.

Figura 17.9 Fluxo de dados da instrução do Power5



- **Alta disponibilidade:** como cada nó no *cluster* é um computador independente, a falha de um nó não significa a perda do serviço. Em muitos produtos, a tolerância a falhas é tratada automaticamente por software.
- **Preço/desempenho superior:** usando a ideia de blocos de construção, é possível montar um *cluster* com poder computacional igual ou maior do que uma única máquina de grande porte, com custo bem menor.



Configurações de *cluster*

Na literatura, os *clusters* são classificados de várias maneiras diferentes. Talvez a classificação mais simples seja baseada no fato de os computadores em um *cluster* compartilharem acesso aos mesmos discos. A Figura 17.10a mostra um *cluster* de dois nós onde a única interconexão é feita por uma ligação da alta velocidade que pode ser usada para troca de mensagens para coordenar as atividade do *cluster*. A ligação pode ser uma LAN compartilhada com outros computadores que não fazem parte do *cluster* ou a ligação pode ser um recurso de interconexão dedicado. No último caso, um ou mais computadores no *cluster* terão a ligação para uma LAN ou WAN para que haja uma conexão entre o *cluster* servidor e sistemas clientes remotos. Observe que, na figura, cada computador é ilustrado como sendo um multiprocessador. Isso não é necessário, porém aumenta o desempenho e a disponibilidade.

Na classificação simples mostrada na Figura 17.10, outra alternativa é um *cluster* de disco compartilhado. Neste caso, geralmente ainda há uma ligação de mensagens entre os nós. Além disso, existe um subsistema de discos que é diretamente ligado a vários computadores dentro do *cluster*. Nesta figura, um subsistema de discos comum é um sistema RAID. O uso de RAID ou de alguma outra tecnologia de discos redundante é comum em *clusters* para que a alta disponibilidade conseguida com a presença de vários computadores não seja comprometida com um disco compartilhado como um ponto único de falha.

Uma ideia mais clara das possibilidades de opções de *clusters* pode ser obtida ao se analisarem alternativas funcionais. A Tabela 17.2 fornece uma classificação útil de acordo com linhas funcionais, as quais analisamos agora.

Um método comum e mais antigo, conhecido como **secundário passivo** (*passive standby*), resume-se a ter um computador lidando com toda a carga de processamento enquanto outro permanece inativo, pronto

Figura 17.10 Configurações de *clusters*

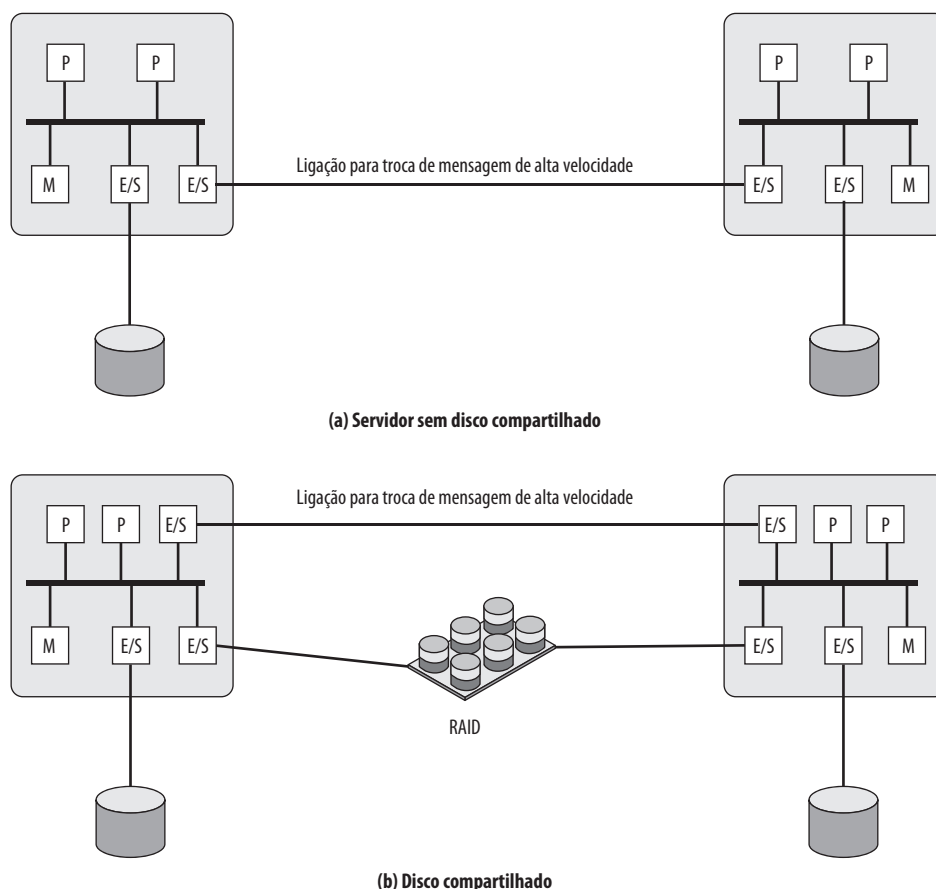


Tabela 17.2 Métodos de *clustering*: benefícios e limitações

Método de <i>clustering</i>	Descrição	Benefícios	Limitações
Secundário passivo (<i>passive standby</i>)	Um servidor secundário assume em caso de falha do servidor primário.	Fácil de implementar.	Custo alto porque o servidor secundário está indisponível para outras tarefas de processamento.
Secundário ativo	O servidor secundário é usado também para tarefas de processamento.	Custo reduzido porque servidores secundários podem ser usados para processamento.	Complexidade aumentada.
Servidores separados	Possuem seus próprios discos. Dados são copiados continuamente do servidor primário para o secundário.	Alta disponibilidade.	Grande sobrecarga de rede e servidores por causa das operações de cópia.
Servidores conectados aos discos	Servidores são ligados aos mesmos discos, mas cada servidor possui seus discos. Se um servidor falha, seus discos são assumidos por outro servidor.	Carga de rede e servidores reduzida por causa da eliminação das operações de cópia.	Normalmente requer espelhamento de discos ou tecnologia RAID para compensar o risco da falha de disco.
Servidores compartilham discos	Vários servidores compartilham simultaneamente o acesso a discos.	Baixa carga de rede e servidores. Risco reduzido de inatividade causada por falha de disco.	Requer software de gerenciamento de bloqueio. Normalmente usado com tecnologia de espelhamento ou RAID.

para assumir em caso de uma falha do primário. Para coordenar as máquinas, o sistema ativo, ou primário, envia periodicamente uma mensagem de reconhecimento para a máquina secundária. Se essas mensagens pararem de chegar, a máquina secundária supõe que o servidor primário falhou e começa a operar. Esta abordagem aumenta a disponibilidade, porém não melhora o desempenho. Além disso, se a única informação trocada entre os dois sistemas é a mensagem de reconhecimento e se os dois sistemas não compartilham discos comuns, então o computador secundário oferece um backup funcional, porém não tem acesso aos bancos de dados gerenciados pelo primário.

O secundário passivo geralmente não é considerada como um *cluster*. O termo *cluster* é reservado para vários computadores interconectados onde todos efetuam processamento ativamente enquanto mantêm a imagem de um sistema único para o mundo externo. O termo **secundário ativo** é frequentemente usado para se referir a esta configuração. Três classificações de *clusters* podem ser identificadas: servidores separados, sem compartilhamento e memória compartilhada.

Em uma abordagem para *clusters*, cada computador é um **servidor separado** com seus próprios discos e não há discos compartilhados entre os sistemas (Figura 17.10a). Este arranjo fornece alto desempenho e disponibilidade. Neste caso, algum tipo de software de gerenciamento ou escalonamento é necessário para atribuir as requisições vindas dos clientes aos servidores para que a carga seja balanceada e alta utilização, alcançada. É desejável que haja a capacidade de tolerância a falhas, o que significa que se um computador falha ao executar uma aplicação, outro computador no *cluster* pode assumir e completar a aplicação. Para que isso aconteça, os dados devem ser constantemente copiados entre os sistemas para que cada um tenha acesso aos dados atuais dos outros sistemas. A sobrecarga dessa troca de dados garante a alta disponibilidade a custo de uma penalidade de desempenho.

Para reduzir a sobrecarga de comunicação, a maioria dos *clusters* consiste agora de servidores conectados aos discos comuns (Figura 17.10b). Em uma variação desta abordagem, chamada de **sem compartilhamento**, os discos comuns são particionados em volumes e cada volume é propriedade de um único computador. Se esse computador falha, o *cluster* deve ser reconfigurado para que algum outro computador tenha posse dos volumes do computador que falhou.

É possível também fazer com que vários computadores compartilhem os mesmos discos ao mesmo tempo (chamada abordagem de **disco compartilhado**), para que cada computador tenha acesso a todos os volumes de todos os discos. Esta abordagem requer o uso de algum tipo de recurso de bloqueio para garantir que os dados possam ser acessados apenas por um computador por vez.



Questões sobre projeto dos sistemas operacionais

O aproveitamento completo de uma configuração de um hardware de *cluster* requer alguns aprimoramentos em sistemas operacionais voltados para sistemas únicos.

GERENCIAMENTO DE FALHAS Como as falhas são gerenciadas pelo *cluster* que depende do método de *clustering* usado (Tabela 17.2). Em geral, duas abordagens podem ser usadas para lidar com falhas: *clusters* de alta disponibilidade e *clusters* com tolerância a falhas. Um *cluster* com alta disponibilidade provê uma alta probabilidade de que todos os recursos estejam em funcionamento. Caso ocorra uma falha, como um desligamento de sistema ou perda de um volume de disco, então as consultas em progresso são perdidas. Qualquer consulta perdida, se tentada novamente, será executada por um computador diferente no *cluster*. No entanto, o sistema operacional de *cluster* não dá garantia alguma sobre o estado de transações executadas parcialmente. Isso deve ser tratado em nível de aplicações.

Um *cluster* com tolerância a falhas garante que todos os recursos estejam sempre disponíveis. Isso é alcançado com o uso de discos compartilhados redundantes e mecanismos para retornar as transações não encerradas e encerrar transações completadas.

A função de trocar as aplicações e recursos de dados de um sistema que falhou para um sistema alternativo no *cluster* é conhecida como **failover** (*recuperação de falhas*). Uma função relacionada é a restauração de aplicações e recursos de dados para o sistema original quando o mesmo for consertado; isto é chamado de **failback** (*retorno à operação*). O *failback* pode ser automatizado, mas isso é desejável apenas se o problema é corrigido realmente e é pouco provável que ocorra novamente. Caso contrário, o *failback* automático pode fazer com que os recursos que falharam sejam passados entre os computadores para lá e para cá, resultando em problemas de desempenho e restauração.

BALANCEAMENTO DE CARGA Um *cluster* requer uma capacidade eficiente para balancear a carga entre computadores disponíveis. Isto inclui o requisito de que o *cluster* seja incrementalmente escalável. Quando um novo computador é adicionado ao *cluster*, o recurso de balanceamento de carga deve automaticamente incluir esse computador no agendamento de aplicações. Mecanismos de *middleware* precisam reconhecer que serviços podem aparecer em diferentes membros do *cluster* e muitos podem migrar de um membro para outro.

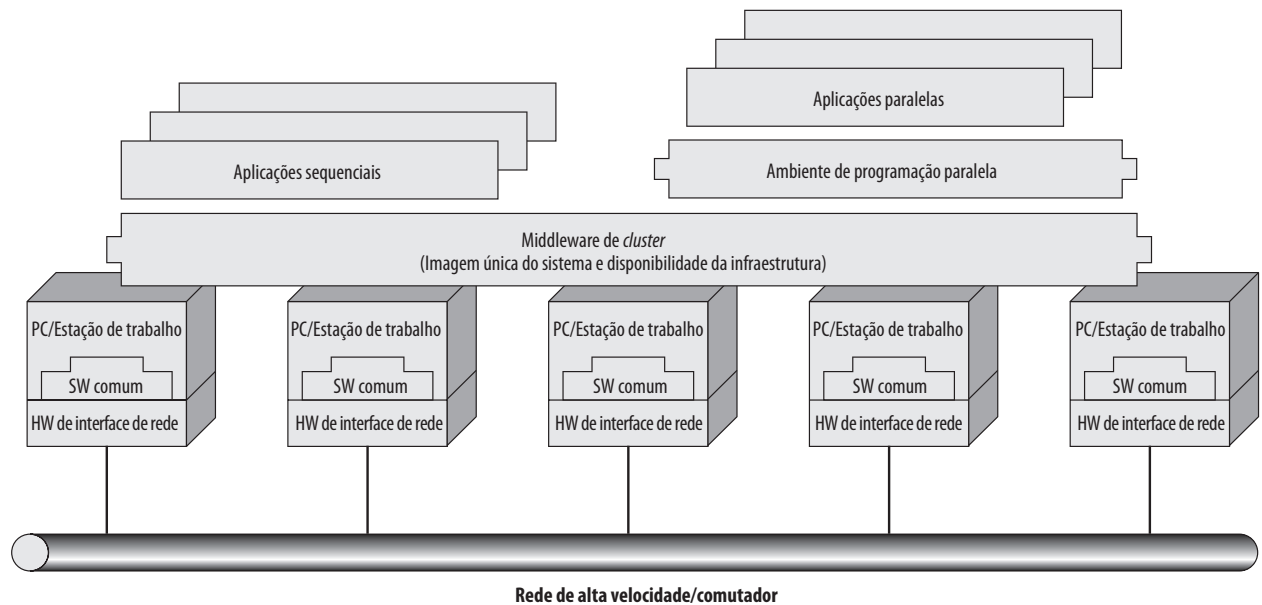
COMPUTAÇÃO PARALELA Em alguns casos, o uso eficiente de um *cluster* requer executar software de uma única aplicação em paralelo. Kapp (2000¹) lista três abordagens gerais para o problema:

- **Compilação paralela:** uma compilação paralela determina, em tempo de compilação, quais partes de uma aplicação podem ser executadas em paralelo. Elas são então separadas para serem atribuídas a diferentes computadores no *cluster*. O desempenho depende da natureza do problema e quão bem o computador é projetado. Em geral, tais compiladores são difíceis de desenvolver.
- **Aplicações paralelas:** nesta abordagem, o programador escreve a aplicação desde o começo para ser executada em um *cluster* e utiliza passagem de mensagens para mover dados, conforme necessário, entre os nós do *cluster*. Isso coloca uma grande responsabilidade no programador, mas pode ser a melhor abordagem para explorar *clusters* para algumas aplicações.
- **Computação paramétrica:** esta abordagem pode ser usada se a essência da aplicação for um algoritmo ou um programa que deva ser executado um grande número de vezes, cada vez com um conjunto diferente de condições iniciais ou parâmetros. Um bom exemplo é um modelo de simulação, o qual vai executar um grande número de cenários e depois desenvolver resumos estatísticos dos resultados. Para que esta abordagem seja eficiente, ferramentas de processamento paramétrico são necessárias para organizar, executar e gerenciar os trabalhos de uma forma eficiente.



Arquitetura de um *cluster* computacional

A Figura 17.11 mostra uma típica arquitetura de *cluster*. Os computadores individuais são conectados por alguma LAN de alta velocidade ou hardware de comutação. Cada computador é capaz de operar independentemente. Além disso, uma camada intermediária de software é instalada em cada computador para possibilitar a operação do *cluster*. O *middleware* do *cluster* fornece uma imagem unificada do sistema para o usuário, conhecida como **imagem de sistema único**. O *middleware* é responsável também por fornecer alta disponibilidade pelo balanceamento de carga e respostas a falhas em componentes individuais. Hwang et al. (1999^m) listam estes como os serviços e as funções desejáveis para um *middleware* de *cluster*:

Figura 17.11 Arquitetura de um *cluster* computacional (Buyya, 1999ⁿ)

- **Ponto de entrada único:** o usuário efetua login no *cluster* em vez de fazê-lo em um computador individual.
- **Hierarquia única de arquivos:** o usuário vê uma hierarquia única de diretórios de arquivos abaixo do mesmo diretório raiz.
- **Ponto de controle único:** há uma estação de trabalho padrão usada para gerenciamento e controle do *cluster*.
- **Rede virtual única:** qualquer nó pode acessar qualquer outro ponto no *cluster*, mesmo que a configuração atual do *cluster* consista em múltiplas redes interconectadas. Há uma operação de rede virtual única.
- **Espaço único de memória:** Memória compartilhada distribuída possibilita que os programas compartilhem variáveis.
- **Sistema único de gerenciamento de trabalhos:** com um agendador de trabalhos do *cluster*, um usuário pode submeter um trabalho sem especificar qual computador executará o trabalho.
- **Interface de usuário única:** uma interface gráfica comum suporta todos os usuários, independentemente da estação de trabalho da qual acessaram o *cluster*.
- **Espaço de E/S único:** qualquer nó pode acessar remotamente qualquer periférico de E/S ou dispositivo de disco sem conhecer a sua localização física.
- **Espaço único de processos:** um esquema uniforme de identificação de processos é usado. Um processo em qualquer nó pode criar ou se comunicar com qualquer outro processo em um nó remoto.
- **Pontos de verificação:** esta função periodicamente salva o estado dos processos e resultados computacionais intermediários para permitir recuperação em caso de falhas.
- **Migração de processos:** esta função habilita o balanceamento de carga.

Os quatro últimos itens da lista anterior aprimoram a disponibilidade do *cluster*. Os itens restantes se preocupam em fornecer uma imagem única do sistema.

Retornando à Figura 17.11, um *cluster* incluirá também ferramentas de software para habilitar a execução eficiente de programas que são capazes de efetuar execução paralela.



Servidores blade

Uma implementação comum da abordagem de *clusters* é o servidor blade. Um servidor blade é uma arquitetura de servidor que hospeda múltiplos módulos servidores (“blades”) em um chassi único. Ela é usada amplamente em centro de armazenamento de dados para economizar espaço e melhorar o gerenciamento de sistemas.

Independentes ou montados no rack, os chassis fornecem fonte de energia e cada blade possui processador, memória e disco rígido próprios.

Um exemplo da aplicação é mostrado na Figura 17.12, retirada de Nowell, Vusirikala e Hays (2007^o). A tendência em grandes centro de armazenamento de dados, com vários bancos de servidores *blade*, é a implementação de portas de 10 Gbps em servidores individuais para lidar com grande tráfego de multimídia fornecidos por esses servidores. Tais arranjos estressam os comutadores Ethernet necessários para interconectar grande número de servidores. Comutadores Ethernet de 100 Gbps são implementados dentro de centro de armazenamento de dados, assim como as conexões de grande alcance para redes corporativas que interligam prédios, campi e outros.



Clusters comparados a SMP

Tanto clusters quanto multiprocessadores simétricos fornecem uma configuração com múltiplos processadores para suportar aplicações com grande demanda. As duas soluções estão disponíveis comercialmente, embora esquemas SMP estejam presentes há mais tempo.

A principal força da abordagem SMP é que um SMP é mais fácil de gerenciar e configurar do que um *cluster*. O SMP é muito mais próximo ao modelo original de processador único para o qual quase todas as aplicações são escritas. A principal alteração requerida quando se muda de um processador único para SMP é com a função de agendamento. Outro benefício de SMP é que ele geralmente ocupa menos espaço físico e consome menos energia do que um *cluster* comparável. Um último benefício importante é que os produtos SMP estão bem estabelecidos e são estáveis.

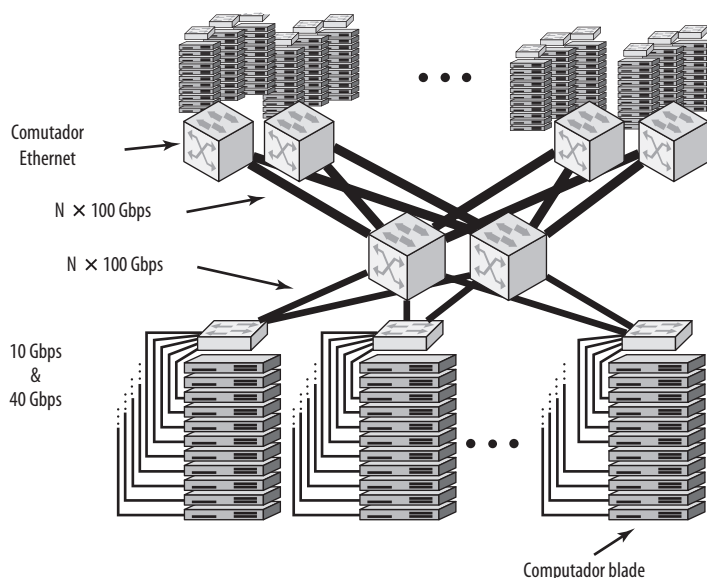
No entanto, ao decorrer do tempo, as vantagens da abordagem de *cluster* provavelmente resultarão na dominação de *clusters* no mercado de servidores de alto desempenho. *Clusters* são muito superiores a SMPs em termos de escalabilidade incremental e absoluta. Eles são superiores também em termos de disponibilidade, porque todos os componentes podem se tornar altamente redundantes.



17.6 Acesso não uniforme à memória

Em termos de produtos comerciais, duas abordagens comuns para fornecer sistemas com vários processadores para suportar aplicações são SMPs e *clusters*. Por alguns anos, outra abordagem, conhecida como acesso não uniforme à memória (NUMA), foi assunto de pesquisa e produtos comerciais NUMA estão disponíveis agora.

Figura 17.12 Exemplo de configuração de Ethernet de 100 Gbps para processamento massivo do servidor blade



Antes de prosseguir, devemos definir alguns termos encontrados frequentemente na literatura sobre NUMA:

- **Acesso uniforme à memória (UMA, do inglês *Uniform memory access*):** todos os processadores têm acesso a todas as partes da memória principal usando leituras e escritas. O tempo de acesso à memória de um processador para todas as regiões da memória é o mesmo. Os tempos de acesso de processadores diferentes são os mesmos. A organização SMP discutida nas seções 17.2 e 17.3 é UMA.
- **Acesso não uniforme à memória (NUMA):** todos os processadores têm acesso a todas as partes da memória principal usando leituras e escritas. O tempo de acesso à memória de um processador difere dependendo de qual região da memória está sendo acessada. Isto é verdade para todos os processadores; no entanto, para processadores diferentes, as regiões de memória mais lentas e mais rápidas diferem.
- **NUMA com coerência de cache (CC-NUMA):** um sistema NUMA no qual a coerência de cache é mantida entre caches de vários processadores.

Um sistema NUMA sem coerência de cache é mais ou menos equivalente a um *cluster*. Os produtos comerciais que receberam muita atenção recentemente são sistemas CC-NUMA, os quais são bastante diferentes de SMPs e *clusters*. Em geral, mas infelizmente nem sempre, tais sistemas são referidos na literatura comercial como sistemas CC-NUMA. Esta seção dedica-se apenas a estes sistemas.



Motivação

Com um sistema SMP, há um limite prático do número de processadores que podem ser usados. Um esquema de cache eficiente reduz o tráfego de barramento entre qualquer um dos processadores e a memória principal. À medida que aumenta o número de processadores, esse tráfego de barramento também aumenta. Além disso, o barramento é usado para trocar sinais de coerência de cache, o que piora ainda mais a situação. Em algum ponto, o barramento torna-se um gargalo de desempenho. A degradação do desempenho parece limitar o número de processadores em uma configuração SMP para algo em torno de 16 até 64 processadores. Por exemplo, o Power Challenge SMP da Silicon Graphics é limitado a 64 processadores R10000 em um sistema único; além desse número, o desempenho degrada substancialmente.

O limite de processadores em um SMP é uma das principais motivações por trás do desenvolvimento de sistemas *clusters*. No entanto, em um *cluster*, cada nó possui sua memória principal privada, as aplicações não enxergam uma memória global maior. Na prática, a coerência é mantida em software em vez de em hardware. Esta granularidade da memória afeta o desempenho e, para alcançar o desempenho máximo, o software deve ser adaptado para esse ambiente. Uma abordagem para alcançar multiprocessamento em grande escala e ao mesmo tempo manter as características de SMP é NUMA. Por exemplo, o sistema Silicon Graphics Origin NUMA é projetado para suportar até 1024 processadores MIPS R10000 (WHITNEY et al., 1997^p) e o sistema Sequent NUMA-Q é projetado para suportar até 252 processadores Pentium II (LOVETT e CLAPP, 1996^q).

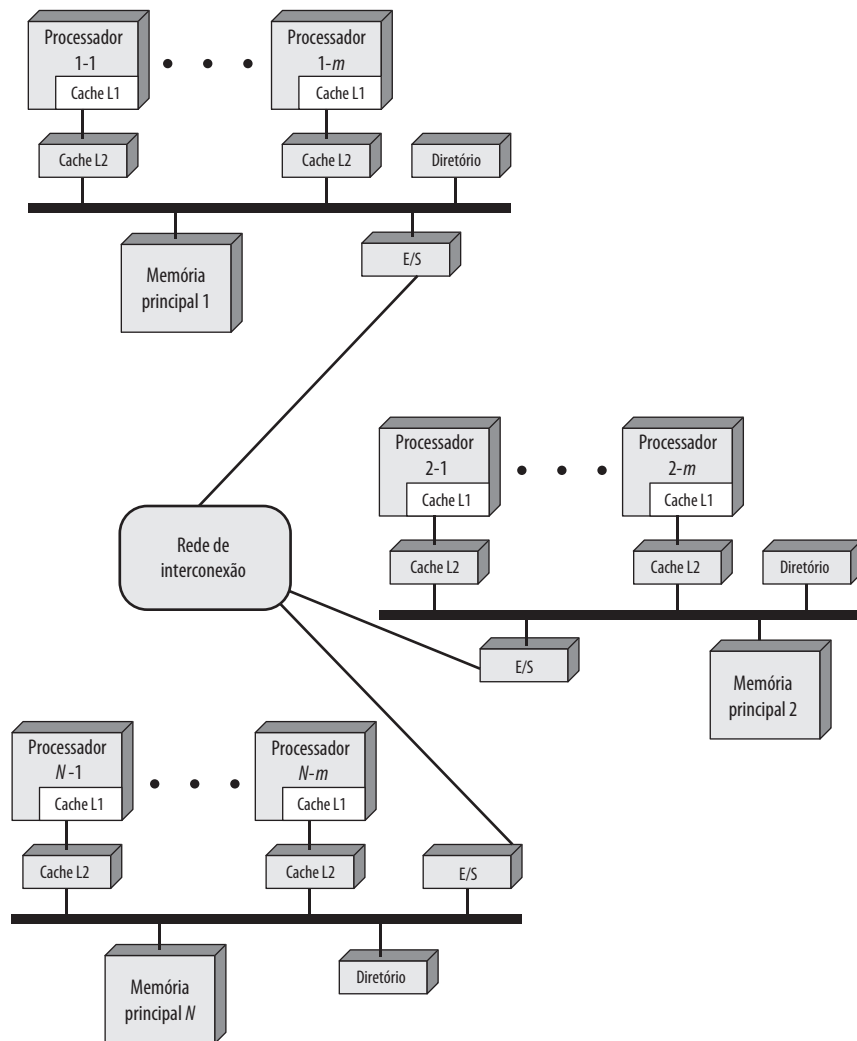
O objetivo com a NUMA é manter uma memória transparente através do sistema, permitindo ao mesmo tempo vários nós multiprocessadores, cada um com seu próprio barramento ou outro sistema de interconexão interna.



Organização

A Figura 17.13 ilustra uma típica organização CC-NUMA. Existem vários nós independentes e cada um, na prática, é uma organização SMP. Dessa forma, cada nó contém vários processadores, cada um com suas próprias caches L1 e L2, mais a memória principal. O nó é o bloco de construção básico de uma organização CC-NUMA. Por exemplo, cada nó do Silicon Graphics Origin inclui quatro processadores Pentium II. Os nós são interconectados por algum recurso de comunicação, o qual pode ser um comutador, um anel ou algum outro recurso de rede.

Cada nó no sistema CC-NUMA inclui alguma memória principal. No entanto, do ponto de vista dos processadores, existe apenas uma única memória endereçável, onde cada posição possui um endereço único no sistema inteiro. Quando um processador inicia um acesso à memória, se a posição de memória requisitada não estiver na cache do processador, então a cache L2 inicia uma operação de leitura. Se a linha desejada estiver na parte local da memória principal, a linha é obtida pelo barramento local. Se a linha desejada estiver numa parte remota da memória principal, então uma requisição automática é enviada para obter essa linha pela rede de interconexão, entregá-la ao barramento local e depois entregá-la à cache requisitante nesse barramento. Toda esta atividade é automática e transparente ao processador e sua cache.

Figura 17.13 Organização CC-NUMA

Nesta configuração, a coerência de cache é uma preocupação central. Embora as implementações sejam diferentes nos detalhes, em termos gerais podemos dizer que cada nó deve manter algum tipo de diretório que lhe dá uma indicação da posição de várias partes da memória e também a informação sobre o estado da cache. Para ver como este esquema funciona, temos um exemplo retirado de Pfister (1998). Suponha que o processador 3 no nó 2 (P2-3) requisite uma posição de memória 798, a qual está na memória do nó 1. Ocorre a seguinte sequência:

1. P2-3 emite uma requisição de leitura no barramento de monitoração do nó 2 para posição 798.
2. O diretório no nó 2 vê a requisição e reconhece que a posição está no nó 1.
3. O diretório do nó 2 envia uma requisição para o nó 1, a qual o diretório do nó 1 pega.
4. O diretório do nó 1, agindo como suplente de P2-3, requisita o conteúdo de 798, como se fosse um processador.
5. A memória principal do nó 1 responde colocando os dados requisitados no barramento.
6. O diretório do nó 1 pega os dados do barramento.
7. O valor é transferido de volta para o diretório do nó 2.
8. O diretório do nó 2 coloca os dados de volta no barramento do nó 2, agindo como suplente para memória que o guardava originalmente.
9. O valor é apanhado e colocado na cache do P2-3 e entregue ao P2-3.

A sequência anterior explica como os dados são lidos de uma memória remota usando mecanismos de hardware que tornam a transação transparente ao processador. Em cima desses mecanismos, algum tipo de protocolo de coerência de cache é necessário. Vários sistemas diferem nos detalhes de como isso é feito exatamente. Fazemos aqui apenas algumas considerações gerais. Primeiro, como parte da sequência anterior, diretório do nó 1 guarda um registro de que alguma cache remota possui uma cópia da linha contendo a posição 798. Depois, deve haver algum protocolo cooperativo para cuidar das modificações. Por exemplo, se a modificação é feita numa cache, esse fato pode ser enviado via dispersão para outros nós. O diretório de cada nó que recebe essa dispersão pode então determinar se alguma cache local possui essa linha e, se sim, faz com que seja excluída. Se a posição de memória atual está no nó que recebeu a notificação de dispersão, então o diretório do nó precisa manter uma entrada indicando que essa linha de memória está inválida e permanece assim até que ocorra uma escrita de volta. Se outro processador (local ou remoto) requisita a linha inválida, então o diretório local deve forçar uma escrita de volta para atualizar a memória antes de fornecer os dados.



Prós e contras de NUMA

A principal vantagem de um sistema CC-NUMA é que ele pode permitir desempenho eficiente em níveis mais altos de paralelismo do que SMP, sem requerer maiores mudanças no software. Com vários nós NUMA, o tráfego do barramento de qualquer nó individual está limitado a uma demanda com qual o barramento pode lidar. No entanto, se muitos dos acessos à memória forem para nós remotos, o desempenho começa a falhar. Há uma razão para acreditar que essa falha de desempenho pode ser evitada. Primeiro, o uso de caches L1 e L2 é projetado para minimizar todos os acessos à memória, incluindo os remotos. Se uma boa parte do software tiver uma boa localidade temporal, então acessos remotos à memória não devem ser excessivos. Segundo, se o software tiver uma boa localidade espacial e se a memória virtual está em uso, então os dados necessários para uma aplicação residirão em um número limitado de páginas frequentemente usadas que podem ser carregadas inicialmente na memória local da aplicação em execução. Os projetistas do Sequent reportaram que tal localidade espacial aparece, sim, em aplicações representativas (LOVETT e CLAPP 1996^a). Finalmente, o esquema de memória virtual pode ser aprimorado ao incluir no sistema operacional um mecanismo de migração de página que move uma página da memória virtual para um nó que a está usando frequentemente; os projetistas da Silicon Graphics reportaram sucesso com essa abordagem (WHITNEY et al., 1997^l).

Mesmo que a diminuição do desempenho por causa do acesso remoto seja tratada, existem duas outras desvantagens para a abordagem CC-NUMA. Duas em particular são discutidas em detalhes em Pfister (1998^l). Primeiro, o CC-NUMA não se parece transparentemente como um SMP; alterações de software serão necessárias para mover um sistema operacional e aplicações de um sistema SMP para um CC-NUMA. Isso inclui alocação de página, alocação de processos e balanceamento de carga pelo sistema operacional. Uma segunda preocupação é em relação à disponibilidade. Esta é uma questão complexa e depende da implementação exata do sistema CC-NUMA; encontramos uma leitura interessante em Pfister (1998^l).



Simulador de processador vetorial



17.7 Computação vetorial

Embora o desempenho de computadores mainframes de uso geral continue a crescer implacavelmente, ainda existem aplicações que estão além do alcance dos mainframes atuais. Existe uma necessidade por computadores para resolver problemas matemáticos dos processos físicos, tais como os que ocorrem em disciplinas incluindo aerodinâmica, sismologia meteorológica e física atômica, nuclear e dos plasmas.

Normalmente, esses problemas são caracterizados pela necessidade de alta precisão e um programa que efetue repetitivamente operações aritméticas de ponto flutuante em grandes matrizes de números. A maioria desses problemas se enquadra na categoria conhecida como *simulação de campos contínuos*. Basicamente, uma situação

física pode ser descrita por uma superfície ou região em três dimensões (por exemplo, o fluxo de ar adjacente à área de um foguete). Esta superfície é aproximada por uma grade de pontos. Um conjunto de equações diferenciais define o comportamento físico da superfície em cada ponto. As equações são representadas como matriz de valores e coeficientes e a solução envolve operações aritméticas repetidas em matrizes de dados.

Supercomputadores foram desenvolvidos para lidar com esses tipos de problemas. Essas máquinas são normalmente capazes de efetuar bilhões de operações de ponto flutuante por segundo. Diferente dos mainframes, os quais são projetados para multiprogramação e E/S intensivo, o supercomputador é otimizado para o tipo de cálculo numérico que acabamos de descrever.

Ele possui uso limitado e, por causa do seu preço, um mercado limitado. Comparativamente falando, poucas dessas máquinas são operacionais, na maioria das vezes em centros de pesquisa e algumas agências governamentais com funções científicas ou de engenharia. Assim como ocorre com outras áreas de tecnologia computacional, há uma demanda constante para aumentar o desempenho dos supercomputadores. Desta forma, a tecnologia e o desempenho dos supercomputadores continuam a evoluir.

Há outro tipo de sistema que foi projetado para atender à necessidade de computação vetorial, conhecido como **processador matricial**. Embora um supercomputador seja otimizado para computação vetorial, trata-se de um computador de uso geral, capaz de lidar com processamento escalar e com tarefas comuns de processamento de dados. Processadores matriciais não incluem processamento escalar; eles são configurados como dispositivos periféricos pelos usuários do computador e do mainframe para executarem partes vetoriais dos programas.



Abordagens para computação vetorial

O principal ponto para projetar um supercomputador ou um processador matricial é reconhecer que a tarefa principal é executar operações aritméticas de matrizes ou vetores de números de ponto flutuante. Em um computador de propósito geral, isso vai requerer iteração por meio de cada elemento da matriz. Por exemplo, considere dois vetores (matrizes unidimensionais) de números, A e B . Gostaríamos de somá-los e colocar o resultado em C . No exemplo da Figura 17.14, isso requer seis adições separadas. Como poderíamos acelerar esse cálculo? A resposta é introduzir alguma forma de paralelismo.

Várias abordagens foram tomadas para alcançar o paralelismo em computação vetorial. Ilustramos isso com um exemplo. Considere a multiplicação de vetores $C = A \times B$, onde A , B e C são matrizes $N \times N$. A fórmula para cada elemento de C é

$$c_{i,j} = \sum_{k=1}^N a_{i,k} \times b_{k,j}$$

onde A , B e C possuem elementos $a_{i,j}$, $b_{i,j}$ e $c_{i,j}$ respectivamente. A Figura 17.15a mostra um programa FORTRAN para esse cálculo que pode ser executado em um processador escalar comum.

Uma abordagem para melhorar o desempenho pode ser referida como *processamento vetorial*. Isto supõe que seja possível operar em um vetor de dados unidimensional. A Figura 17.15b é um programa FORTRAN com uma nova forma de instrução que permite que computação vetorial seja especificada. A notação ($J = 1, N$) indica que as operações em todos os índices J no dado intervalo serão executadas como uma única operação.

Figura 17.14 Exemplo de soma de vetores

$\begin{bmatrix} 1,5 \\ 7,1 \\ 6,9 \\ 100,5 \\ 0 \\ 59,7 \end{bmatrix}$	+	$\begin{bmatrix} 2,0 \\ 39,7 \\ 1000,003 \\ 11 \\ 21,1 \\ 19,7 \end{bmatrix}$	=	$\begin{bmatrix} 3,5 \\ 46,8 \\ 1006,093 \\ 111,5 \\ 21,1 \\ 79,4 \end{bmatrix}$
A		B		C

Figura 17.15 Multiplicação de matrizes ($C = A \times B$)

```

DO 100 I = 1, N
DO 100 J = 1, N
C(I, J) = 0.0
DO 100 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J)
100 CONTINUE

```

(a) Processamento escalar

```

DO 100 I = 1, N
C(I, J) = 0.0 (J = 1, N)
DO 100 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J) (J = 1, N)
100 CONTINUE

```

(b) Processamento vetorial

```

DO 50 J = 1, N - 1
FORK 100
50 CONTINUE
J = N
100 DO 200 I = 1, N
C(I, J) = 0.0
DO 200 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J)
200 CONTINUE
JOIN IN

```

(c) Processamento paralelo

O programa da Figura 17.15b indica que todos os elementos da i -ésima linha serão calculados em paralelo. Cada elemento da linha é um somatório e os somatórios (pelos K) são feitos serialmente em vez de em paralelo. Mesmo assim, apenas N^2 multiplicações de vetores são necessárias para este algoritmo se comparado com N^3 multiplicações escalares para o algoritmo escalar.

Outra abordagem, *processamento paralelo*, é ilustrada na Figura 17.15c. Esta abordagem assume que tenhamos N processadores independentes que podem funcionar em paralelo. Para utilizar processadores de modo eficiente, temos que dividir de alguma forma os cálculos em vários processadores. Duas primitivas são usadas. A primitiva **FORK n** faz com que um processo independente seja iniciado na posição n . Ao mesmo tempo, o processo original continua a execução na instrução que vem imediatamente depois de **FORK**. Cada execução de um **FORK** gera um processo novo. A instrução **JOIN** é basicamente o inverso de **FORK**. A cláusula **JOIN N** faz com que N processos independentes sejam unidos em um que continua a execução na instrução que segue **JOIN**. O sistema operacional deve coordenar essa junção e assim a execução não continua até que todos os N processos tenham alcançado a instrução **JOIN**.

O programa na Figura 17.15c é escrito para imitar o comportamento de um programa de processamento vetorial. No programa de processamento paralelo, cada coluna de C é calculada por um processo separado. Assim, os elementos em uma dada linha de C são computados em paralelo.

A discussão anterior descreve abordagens para computação vetorial em termos lógicos ou arquiteturais. Vamos focar agora em uma consideração sobre os tipos de organização de processadores que podem ser usados para implementar essas abordagens. Uma grande variedade de organizações foi e está sendo praticada. Três principais categorias se destacam:

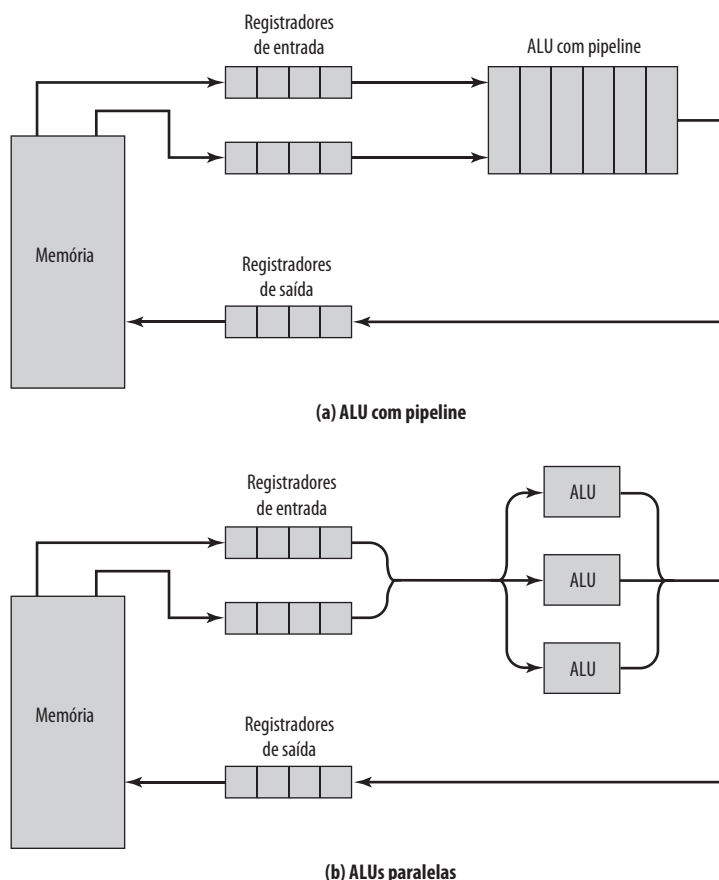
- ALU com pipeline.
- ALUs paralelas.
- Processadores paralelos.

A Figura 17.16 ilustra as duas primeiras abordagens. Já discutimos o pipeline no Capítulo 12. Aqui o conceito é estendido para operação da ALU. Como as operações de ponto flutuante são bastante complexas, há uma oportunidade para decompor uma operação de ponto flutuante em estágios, para que estágios diferentes possam operar em conjuntos de dados diferentes concorrentemente. Isso é ilustrado na Figura 17.17a. Adição de ponto flutuante é quebrada em quatro estágios (veja Figura 9.22): comparar, deslocar, adicionar e normalizar. Um vetor de números é apresentado subsequentemente para o primeiro estágio. À medida que o processamento procede, quatro conjuntos diferentes de números serão processados concorrentemente no pipeline.

Deve estar claro que esta organização é adequada para processamento vetorial. Para perceber isso, considere o pipeline de instruções descrito no Capítulo 12. O processador passa por um ciclo repetitivo de ler e processar instruções. Na ausência de desvios, o processador está continuamente obtendo instruções de posições sequenciais. Consequentemente, o pipeline é mantido cheio e economia em tempo é conseguida. De forma semelhante, uma ALU com pipeline irá economizar tempo apenas se for alimentada com um fluxo de dados a partir das posições sequenciais. Uma operação única, isolada do ponto flutuante, não é acelerada por um pipeline. A aceleração é conseguida quando um vetor de operandos é apresentado para a ALU. A unidade de controle envia os dados através da ALU até que o vetor inteiro seja processado.

A operação do pipeline pode ser melhorada ainda mais se os elementos do vetor estiverem disponíveis nos registradores, e não na memória. De fato, isso é sugerido na Figura 17.16a. Os elementos de cada operando do vetor são carregados como um bloco em um registrador vetorial, o qual é simplesmente um grande banco de registradores idênticos. O resultado é guardado também em um registrador vetorial. Desta forma, a maioria das operações envolve apenas o uso de registradores, e apenas as operações de leitura e escrita e começo e fim de uma operação vetorial requerem acesso à memória.

Figura 17.16 Abordagens para computação vetorial

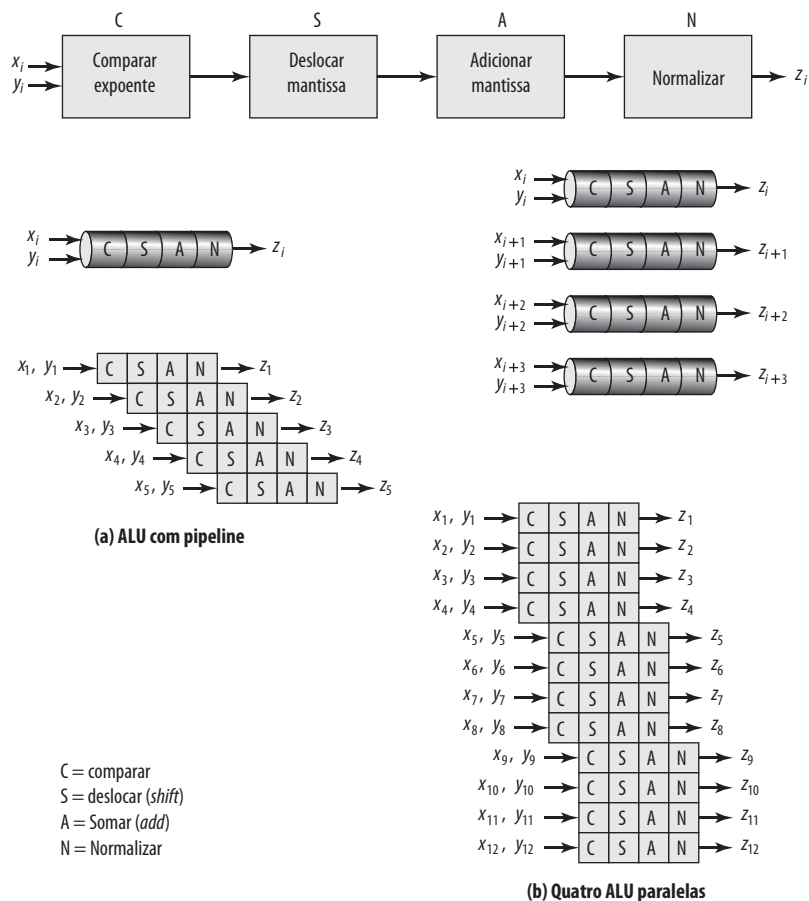


O mecanismo ilustrado na Figura 17.17 pode ser referido como *pipeline dentro de uma operação*. Isto é, temos uma única operação aritmética (por exemplo, $C = A + B$) que será aplicada aos operandos do vetor, e o uso do pipeline permite que vários elementos do vetor sejam processados em paralelo. Este mecanismo pode ser aumentado com *pipeline através de operações*. Neste último caso, há uma sequência de operações vetoriais aritméticas e o pipeline de instruções é usado para acelerar o processamento. Uma abordagem para isso, conhecida como **encadeamento (chaining)**, é encontrada nos supercomputadores Cray. A regra básica para encadeamento é: uma operação vetorial pode começar assim que o primeiro elemento do(s) vetor(es) do operando estiver disponível e a unidade funcional (por exemplo, adicionar, subtrair, multiplicar, dividir) estiver livre. Basicamente, o encadeamento faz com que a emissão de resultados de uma unidade funcional seja alimentada em outra unidade funcional e assim por diante. Se os registradores vetoriais forem usados, resultados intermediários não precisam ser armazenados em memória e podem ser usados até antes que a operação vetorial que os criou execute até a conclusão.

Por exemplo, quando computar $C = (s \times A) + B$, onde A , B e C são vetores e s é um escalar, o Cray pode executar três instruções ao mesmo tempo. Os elementos obtidos entram imediatamente no multiplicador com pipeline, os produtos são enviados para um somador com pipeline e as somas são guardadas em um registrador vetorial assim que o somador as completa:

1. Carregar vetor $A \rightarrow$ Registrador vetorial (VR1)
2. Carregar vetor $B \rightarrow$ VR2
3. Multiplicar vetor $s \times$ VR1 $\rightarrow$ VR3
4. Adicionar vetor VR3 + VR2 $\rightarrow$ VR4
5. Armazenar vetor VR4 $\rightarrow C$

Figura 17.17 Processamento com pipeline de operações de ponto flutuante



As instruções 2 e 3 podem ser encadeadas porque elas envolvem posições de memória e registradores diferentes. A instrução 4 precisa dos resultados das instruções 2 e 3, mas pode ser encadeada com elas também. Assim que os primeiros elementos dos registradores vetoriais 2 e 3 estiverem disponíveis, a operação na instrução 4 pode começar.

Outra forma de alcançar processamento vetorial é pelo uso de múltiplas ALUs em um único processador, sob controle de uma única unidade de controle. Neste caso, a unidade de controle roteia os dados para as ALUs para que elas possam funcionar em paralelo. É possível também usar pipeline em cada ALUs paralela. Isso é ilustrado na Figura 17.17b. O exemplo mostra um caso onde quatro ALUs operam em paralelo.

Assim como acontece com organização de pipeline, uma organização paralela de ALUs é adequada para processamento vetorial. A unidade de controle encaminha elementos vetoriais para ALUs no modo *round-robin* até que todos os elementos sejam processados. Este tipo de organização é mais complexo que uma única ALU CPI.

Finalmente, o processamento vetorial pode ser alcançado com uso de vários processadores paralelos. Neste caso, é necessário quebrar a tarefa em vários processos para serem executados em paralelo. Esta organização é eficiente apenas se tivessem software e hardware eficientes para a coordenação de processadores paralelos.

Podemos expandir a nossa taxonomia da Seção 17.1 para refletir essas estruturas novas, conforme mostrado na Figura 17.18. Organizações computacionais podem ser diferenciadas pela presença de uma ou mais unidades de controle. Várias unidades de controle implicam em vários processadores. Segundo a nossa discussão anterior, se vários processadores podem funcionar de forma cooperativa em uma determinada tarefa, eles podem ser chamados de *processadores paralelos*.

O leitor deve estar ciente de alguma terminologia infeliz bastante encontrada na literatura. O termo *processador vetorial* é frequentemente igualado a uma organização de ALU com pipeline, embora uma organização paralela de ALUs seja projetada também para processamento vetorial e, conforme já discutimos, uma organização de processadores paralelos também pode ser projetada para processamento vetorial. O *processamento matricial* às vezes é usado para se referir a ALUs paralela, embora, novamente, qualquer uma das três organizações seja otimizada para processamento de matrizes. Para piorar ainda mais as coisas, *processador matricial* normalmente refere-se a um processador auxiliar anexado a um processador de uso geral e usado para executar computação vetorial. Um processador matricial pode usar abordagem de ALUs paralelas ou com pipeline.

No momento, a organização de ALUs com pipeline domina o mercado. Os sistemas com pipeline são menos complexos que as duas outras abordagens. A sua unidade de controle e o projeto do sistema operacional são bem desenvolvidos para alcançar alocação de recursos eficiente e alto desempenho. O restante desta seção é dedicado para uma análise mais detalhada dessa abordagem, usando um exemplo específico.

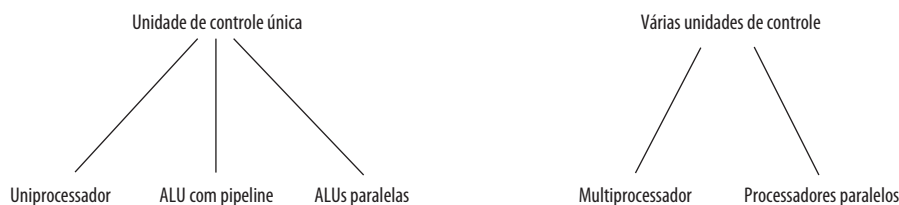


Recurso vetorial do IBM 3090

Um bom exemplo de uma organização de ALU com pipeline para processamento vetorial é o recurso vetorial desenvolvido para arquitetura IBM 370 e implementado nas séries 3090 de alto nível (PADEGS et al., 1988; TUCKER, 1987). Este recurso é um opcional adicional ao sistema básico, mas é altamente integrado a ele e se assemelha aos recursos vetoriais encontrados em supercomputadores, como os da família Cray.

O recurso da IBM faz uso de uma série de registradores vetoriais. Cada registrador é, na verdade, um banco de registradores escalares. Para calcular a soma de vetores $C = A + B$, os vetores A e B são carregados em dois registradores vetoriais. Os dados desses registradores passam pela ALU o mais rápido possível e os resultados são guardados em

Figura 17.18 Uma taxonomia de organizações computacionais



um terceiro registrador vetorial. A sobreposição computacional e a leitura de dados de entrada nos registradores em um bloco resultam em uma aceleração significativa em relação a uma operação de uma ALU comum.

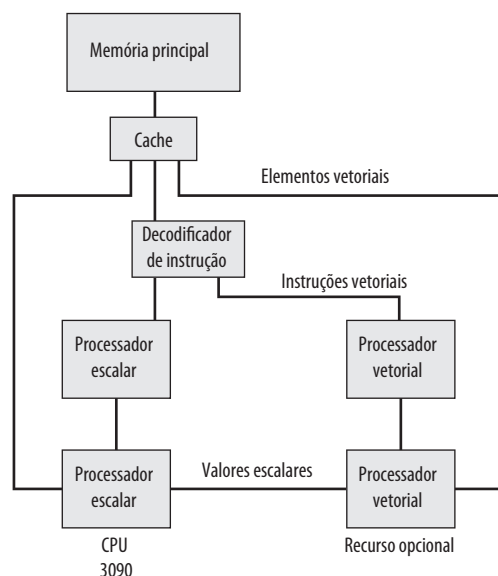
ORGANIZAÇÃO Arquitetura vetorial da IBM e ALUs vetoriais com pipeline similares fornecem um aumento no desempenho em relação a laços de instruções aritméticas escalares de três maneiras:

- A estrutura fixa e predeterminada de dados do vetor permite que instruções guardadas dentro do laço sejam substituídas por operações de máquina internas mais rápidas (de hardware ou microcódigo).
- O acesso a dados e operações aritméticas em vários elementos vetoriais sucessivos pode proceder concorrentemente por meio da sobreposição de tais operações em um projeto com pipeline ou por meio da execução de operações de múltiplos elementos em paralelo.
- O uso de registradores vetoriais para resultados intermediários evita referências adicionais de armazenamento.

A Figura 17.19 mostra a organização geral do recurso vetorial. Embora ele seja visto como um adicional físico separado do processador, a sua arquitetura é uma extensão da arquitetura do System/370 e é compatível com ela. O recurso vetorial é integrado na arquitetura do System/370 de seguintes maneiras:

- Instruções existentes de System/370 são usadas para todas as operações escalares.
- Operações aritméticas em elementos vetoriais individuais produzem exatamente o mesmo resultado como instruções escalares do System/370 correspondentes. Por exemplo, uma decisão de projeto a respeito da definição do resultado em uma operação DIVIDE de ponto flutuante. O resultado deve ser exato, como é para divisão escalar de ponto flutuante, ou uma aproximação deve ser aceita para que seja permitida uma implementação de velocidade maior, mas que pode introduzir às vezes um erro em uma ou mais posições de bits de ordem mais baixa? A decisão foi feita para manter a compatibilidade total com a arquitetura de System/370 a custo de uma pequena degradação do desempenho.
- Instruções vetoriais podem ser interrompidas e sua execução pode ser continuada do ponto da interrupção depois que a ação adequada foi tomada, de uma forma compatível com esquema de interrupção de programa do System/370.
- Exceções aritméticas são as mesmas que, ou extensões de, exceções para instruções aritméticas escalares do System/370 e rotinas de ajustes similares podem ser usadas. Para acomodar isso, um índice de interrupção vetorial é empregado e indica a posição dentro de um registrador vetorial que é afetado por uma exceção

Figura 17.19 IBM 3090 com recurso vetorial



(por exemplo, *overflow*). Desta forma, quando a execução da instrução vetorial continua, o lugar adequado em um registrador vetorial é acessado.

- Os dados vetoriais residem em armazenamento vetorial, com falhas de página sendo tratadas de forma padrão.

Este nível de integração fornece vários benefícios. Os sistemas operacionais existentes podem suportar o recurso vetorial com pequenas extensões. As aplicações existentes, compiladores e outros softwares podem ser executados sem mudanças. O software que poderia obter vantagem do recurso vetorial pode ser modificado conforme desejado.

REGISTRADORES O principal ponto em projetar um recurso vetorial é se os operandos são localizados em registradores ou memória. A organização da IBM é chamada de *registrador para registrador*, porque os operandos vetoriais de saída e entrada podem ser guardados em registradores vetoriais. Esta abordagem é usada também no supercomputador Cray. Uma abordagem alternativa, usada em máquinas Control Data, é obter operandos diretamente da memória. A principal desvantagem do uso de registradores vetoriais é que programador ou compilador deve considerá-los para um bom desempenho. Por exemplo, suponha que o tamanho dos registradores vetoriais seja K e o tamanho dos vetores a serem processados seja $N > K$. Neste caso, um laço de vetor deve ser executado, no qual a operação é executada em K elementos ao mesmo tempo e o laço é repetido N/K vezes. A principal vantagem do registrador vetorial é que a operação é desacoplada da memória principal mais lenta e, em vez disso, ocorre principalmente em registradores.

A aceleração que pode ser alcançada com uso de registradores é demonstrada na Figura 17.20. A rotina FORTRAN multiplica o vetor A pelo vetor B para produzir o vetor C, onde cada vetor possui uma parte real (AR, BR, CR) e uma

Figura 17.20 Programas alternativos para cálculos de vetores

ROTINA FORTRAN:

```
DO 100 J = 1, 50
  CR(J) = AR(J) * BR(J) - AI(J) * BI(J)
100  CI(J) = AR(J) * BI(J) + AI(J) * BR(J)
```

Operação	Ciclos
AR(J) * BR(J) → T1(J)	3
AI(J) * BI(J) → T2(J)	3
T1(J) - T2(J) → CR(J)	3
AR(J) * BI(J) → T3(J)	3
AI(J) * BR(J) → T4(J)	3
T3(J) + T4(J) → CI(J)	3
Total	18

(a) Memórias para memória

Operação	Ciclos
AR(J) → V1(J)	1
V1(J) * BR(J) → V2(J)	1
AI(J) → V3(J)	1
V3(J) * BI(J) → V4(J)	1
V2(J) - V4(J) → V5(J)	1
V5(J) → CR(J)	1
V1(J) * BI(J) → V6(J)	1
V4(J) * BR(J) → V7(J)	1
V6(J) + V7(J) → V8(J)	1
V8(J) → CI(J)	1
Total	10

(c) Memórias para registrador

Vi = registradores vetoriais
AR, BR, AI, BI = operandos em memória
Ti = posições temporárias em memória

Operação	Ciclos
AR(J) → V1(J)	1
BR(J) → V2(J)	1
V1(J) * V2(J) → V3(J)	1
AI(J) → V4(J)	1
BI(J) → V5(J)	1
V4(J) * V5(J) → V6(J)	1
V3(J) - V6(J) → V7(J)	1
V7(J) → CR(J)	1
V1(J) * V5(J) → V8(J)	1
V4(J) * V2(J) → V9(J)	1
V8(J) + V9(J) → V0(J)	1
V0(J) → CI(J)	1
Total	12

(b) Registrador para registrador

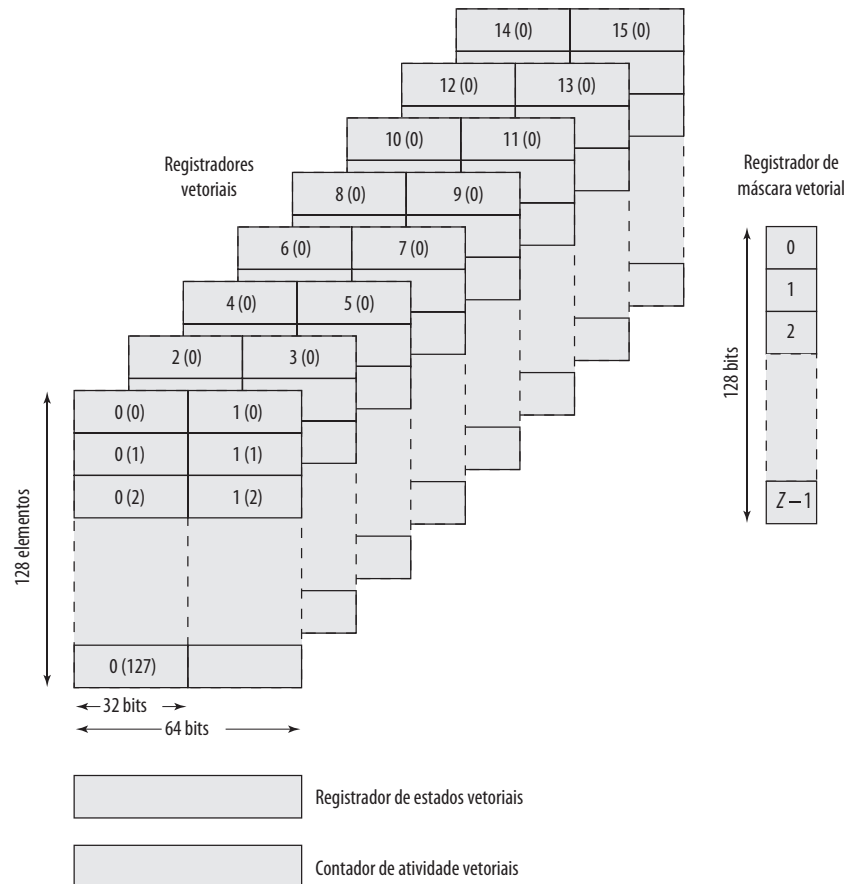
Operação	Ciclos
AR(J) → V1(J)	1
V1(J) * BR(J) → V2(J)	1
AI(J) → V3(J)	1
V2(J) - V3(J) * BI(J) → V2(J)	1
V2(J) → CR(J)	1
V1(J) * BI(J) → V4(J)	1
V4(J) + V3(J) * BR(J) → V5(J)	1
V5(J) → CI(J)	1
Total	8

(d) Instrução composta

parte imaginária (AI, BI, CI). O 3090 pode executar um acesso ao armazenamento principal por ciclo de processador, ou clock (leitura ou escrita); possui registradores que podem sustentar dois acessos para leitura e um para escrita por ciclo; e produz um resultado por ciclo na sua unidade aritmética. Vamos supor o uso de instruções que podem especificar dois operandos fontes e um resultado.⁵ A Figura 17.20a mostra que, com instruções memória-para-memória, cada iteração do processamento requer um total de 18 ciclos. Com arquitetura puramente registrador-para-registrador (Figura 17.20b), esse tempo é reduzido para 12 ciclos. É claro que, com operações registrador-para-registrador as grandezas vetoriais devem ser carregadas nos registradores vetoriais antes do processamento e armazenadas em memória logo depois. Para vetores grandes, esta punição fixa é relativamente pequena. A Figura 17.20c mostra que a habilidade de especificar operandos de memória e de registrador em uma instrução reduz o tempo ainda mais para 10 ciclos por iteração. Este último tipo de instrução é incluído na arquitetura vetorial.⁶

A Figura 17.21 ilustra os registradores que são parte do recurso vetorial do IBM 3090. Existem 16 registradores vetoriais de 32 bits. Os registradores vetoriais também podem ser acoplados para formar oito registradores vetoriais de 64 bits. Qualquer elemento registrador pode guardar um valor inteiro ou de ponto flutuante. Assim, registradores vetoriais podem ser usados para valores inteiros de 32 e 64 bits e para valores de ponto flutuante de 32 e 64 bits.

Figura 17.21 Registradores para recurso vetorial do IBM 3090



5 Para arquitetura 370/390, as únicas instruções de três operandos (instruções de registradores e armazenamento, RS) especificam dois operandos nos registradores e um em memória. Na parte (a) do exemplo, assumimos a existência de instruções de três operandos nas quais todos os operandos estão na memória principal. Isto é feito para propósitos de comparação e, de fato, tal formato de instrução poderia ser escolhido para arquitetura vetorial.

6 Instruções compostas, discutidas a seguir, possibilitam uma redução ainda maior.

A arquitetura especifica que cada registrador contenha de 8 a 512 elementos escalares. A escolha do tamanho atual envolve uma negociação de projeto. O tempo para fazer uma operação vetorial consiste basicamente da sobrecarga para inicializar o pipeline e preencher registradores mais um ciclo por elemento vetorial. Desta forma, o uso de um número grande de elementos registradores reduz a inicialização relativa para uma computação. No entanto, esta eficiência deve ser equilibrada com o tempo adicional necessário para salvar e restaurar registradores vetoriais em uma troca de processos e o custo prático e os limites de espaço. Estas considerações levam ao uso de 128 elementos por registrador na atual implementação do 3090.

Três registradores adicionais são necessários para o recurso vetorial. O registrador de máscara vetorial contém bits da máscara que podem ser usados para selecionar quais elementos nos registradores vetoriais devem ser processados para uma determinada operação. O registrador de estado vetorial contém campos de controle, como contador vetorial que determina quantos elementos nos registradores vetoriais estão para ser processados. O contador de atividade vetorial acompanha o tempo gasto executando instruções vetoriais.

INSTRUÇÕES COMPOSTAS Conforme discutimos anteriormente, execução da instrução pode ser sobreposta usando encadeamento para melhorar o desempenho. Os projetistas do recurso vetorial da IBM escolheram não incluir essa capacidade por vários motivos. A arquitetura do System/370 teria que ser estendida para tratar interrupções complexas (incluindo o seu efeito no gerenciamento da memória virtual) e as alterações correspondentes seriam necessárias no software. Uma questão mais básica foi o custo de incluir os controles adicionais e caminhos de acesso aos registradores no recurso vetorial para um encadeamento geral.

Em vez disso, três operações são fornecidas que combinam, em uma instrução (um *opcode*), as sequências mais comuns em computação vetorial, mais precisamente multiplicação seguida de adição, subtração ou somatório. A instrução memória-para-registrador MULTIPLY-AND-ADD, por exemplo, obtém um vetor da memória, multiplica-o por um vetor de um registrador e adiciona o produto de um terceiro vetor em um registrador. Com uso de instruções compostas MULTIPLY-AND-ADD e MULTIPLY-AND-SUBTRACT no exemplo da Figura 17.20, o tempo total para iteração é reduzido de 10 para 8 ciclos.

Diferente do encadeamento, instruções compostas não requerem o uso de registradores adicionais para armazenamento temporário ou resultados intermediários e requerem um acesso a registrador a menos. Por exemplo, considere a seguinte sequência:

$A \rightarrow VR1$

$VR1 + VR2 \rightarrow VR1$

Neste caso, dois armazenamentos no vetor VR1 são necessários. Na arquitetura da IBM existe uma instrução ADD memória-para-registrador. Com essa instrução, apenas a soma é colocada em VR1. A instrução composta evita também a necessidade para refletir, na descrição do estado da máquina, a execução concorrente de um número de instruções, o que simplifica salvar e restaurar o estado pelo sistema operacional e o tratamento de interrupções.

CONJUNTO DE INSTRUÇÕES A Tabela 17.3 resume as operações aritméticas e lógicas que são definidas para arquitetura vetorial. Além disso, existem instruções de leitura memória-para-registrador e escrita registrador-para-memória. Observe que muitas instruções usam o formato de três operandos. Também, muitas instruções possuem uma série de variantes, dependendo da localização dos operandos. Um operando de origem pode ser um registrador vetorial (V), armazenamento (S) ou um registrador escalar (Q). O alvo é sempre um registrador vetorial, exceto para comparação, cujo resultado vai para o registrador de máscara de vetor. Com todas estas variantes, o número total de *opcodes* (instruções distintas) é 171. No entanto, este número grande não é tão caro para ser implementado como pode parecer. Uma vez a máquina fornecendo as unidades aritméticas e caminhos de dados para alimentar operandos a partir do armazenamento, registradores escalares e registradores vetoriais para pipelines vetoriais, o principal custo de hardware já foi coberto. A arquitetura pode, com pouca diferença no custo, fornecer um conjunto rico de variantes para uso desses registradores e pipelines.

A maioria das instruções da Tabela 17.3 é autoexplicativa. Duas instruções de somatório exigem uma explicação adicional. A operação de acumular agrupa os elementos de um único vetor (ACCUMULATE) ou os elementos do produto de dois vetores (MULTIPLY-AND-ACCUMULATE). Essas instruções apresentam um problema de projeto interessante. Nós gostaríamos de executar essa operação o mais rapidamente possível, obtendo a total vantagem da ALU com pipeline. A dificuldade é que a soma de dois números colocada no

Tabela 17.3 Recurso vetorial de IBM 3090: instruções aritméticas e lógicas

	Tipos de dados						
	Ponto flutuante						
Operação	Longo (long)	Curto (short)	Binário ou Lógico	Localizações dos operandos			
Somar	FL	FS	BI	$V + V \rightarrow V$	$V + S \rightarrow V$	$Q + V \rightarrow V$	$Q + S \rightarrow V$
Subtrair	FL	FS	BI	$V - V \rightarrow V$	$V - S \rightarrow V$	$Q - V \rightarrow V$	$Q - S \rightarrow V$
Multiplicar	FL	FS	BI	$V \times V \rightarrow V$	$V \times S \rightarrow V$	$Q \times V \rightarrow V$	$Q \times S \rightarrow V$
Dividir	FL	FS	–	$V / V \rightarrow V$	$V / S \rightarrow V$	$Q / V \rightarrow V$	$Q / S \rightarrow V$
Comparar	FL	FS	BI	$V . V \rightarrow V$	$V . S \rightarrow V$	$Q . V \rightarrow V$	$Q . S \rightarrow V$
Multiplicar e somar	FL	FS	–	$V + V \times S \rightarrow V$	$V + Q \times V \rightarrow V$	$V + Q \times S \rightarrow V$	
Multiplicar e subtrair	FL	FS	–	$V - V \times S \rightarrow V$	$V - Q \times V \rightarrow V$	$V - Q \times S \rightarrow V$	
Multiplicar e acumular	FL	FS	–	$P + .V \rightarrow V$	$P + .S \rightarrow V$		
Complemento	FL	FS	BI	$-V \rightarrow V$			
Positivo absoluto	FL	FS	BI	$ V \rightarrow V$			
Negativo absoluto	FL	FS	BI	$- V \rightarrow V$			
Máximo	FL	FS	–			$Q . V \rightarrow Q$	
Máximo absoluto	FL	FS	–			$Q . V \rightarrow Q$	
Mínimo	FL	FS	–			$Q . V \rightarrow Q$	
Deslocamento lógico para esquerda	–	–	LO	$.V \rightarrow V$			
Deslocamento lógico para direita	–	–	LO	$.V \rightarrow V$			
And	–	–	LO	$V \& V \rightarrow V$	$V \& S \rightarrow V$	$Q \& V \rightarrow V$	$Q \& S \rightarrow V$
OR	–	–	LO	$V V \rightarrow V$	$V S \rightarrow V$	$Q V \rightarrow V$	$Q S \rightarrow V$
Exclusive-OR	–	–	LO	$V \oplus V \rightarrow V$	$V \oplus S \rightarrow V$	$Q \oplus V \rightarrow V$	$Q \oplus S \rightarrow V$

Explicação:

Tipos de dadosFL = ponto flutuante longo (*long*)FS = ponto flutuante curto (*short*)

BI = inteiro binário

LO = lógico

Localizações dos operandos

V = Registrador vetorial

S = Armazenamento

Q = Escalar (registrador geral ou de ponto flutuante)

P = Somas parciais no registrador vetorial

. = Operação especial

pipeline não está disponível até vários ciclos depois. Assim, o terceiro elemento no vetor não pode ser adicionado à soma dos dois primeiros elementos até que eles passem pelo pipeline inteiro. Para tratar esse problema, os elementos do vetor são adicionados de tal forma que produzem quatro somas parciais. Em particular, elementos 0, 4, 8, 12, ..., 124 são adicionados nessa ordem para produzir a soma parcial 0; elementos 1, 5, 9, 13, ..., 125 para soma parcial 1; elementos 2, 6, 10, 14, ..., 126 para soma parcial 2; e elementos 3, 7, 11, 15, ..., 127 para soma parcial 4. Cada uma dessas somas parciais pode prosseguir pelo pipeline à velocidade máxima porque o atraso do pipeline é de aproximadamente quatro ciclos. Um registrador vetorial separado é usado para guardar as somas parciais. Quando todos os elementos do vetor original forem processados, as quatro somas parciais são totalizadas para produzir o resultado final. O desempenho desta segunda fase não é crítico, porque apenas quatro elementos vetoriais são envolvidos.



17.8 Leitura recomendada e sites Web

Catanzaro (1994^u) analisa os princípios dos multiprocessadores e examina SMP baseados em SPARC em detalhes. SMPs também estão cobertos em bastante detalhes em Stone (1993^v) e Hwang (1993^w).

Milenkovic (2000^x) é uma introdução a algoritmos e técnicas de coerência de cache para multiprocessadores, com ênfase em questões de desempenho. Outra análise das questões referentes à coerência de cache em multiprocessadores é Lilja (1993^d). Tomasevic e Milutinovic (1993^y) contém reimpressão de vários artigos importantes sobre o assunto.

Ungerer, Rubic e Silc (2002^g) é uma análise excelente dos conceitos de processadores *multithread* e chips multiprocessadores. Ungerer, Rubic e Silc (2003^h) fazem uma longa análise de processadores *multithread* propostos e atuais que usam *multithread* explícito.

Um tratamento completo sobre *clusters* pode ser encontrado em Buyya (1999ⁿ) e Buyya (1999^z). Weygant (2001^{aa}) é uma análise menos técnica sobre *clusters*, com bons comentários sobre vários produtos comerciais. Desai et al. (2005^{bb}) descreve a arquitetura do servidor blade da IBM.

Uma boa discussão sobre computação vetorial pode ser encontrada em Stone (1993^x) e Hwang (1993^w).



Sites Web recomendados

IEEE Computer Society Task Force on Cluster Computing: um fórum internacional para promover pesquisa e educação sobre computação em *cluster*.

Principais termos, perguntas de revisão e problemas

Principais termos

Secundário ativo (<i>active standby</i>)	<i>Failover</i>	Protocolo de monitoramento (<i>snoopy</i>)
Coerência de cache	Protocolo MESI	Multiprocessador simétrico (SMP)
<i>Cluster</i>	Multiprocessador	Acesso uniforme à memória (UMA)
Protocolo de diretório	Acesso não uniforme à memória (NUMA)	Uniprocessador
<i>Failback</i>	Secundário passivo (<i>passive standby</i>)	Recurso vetorial

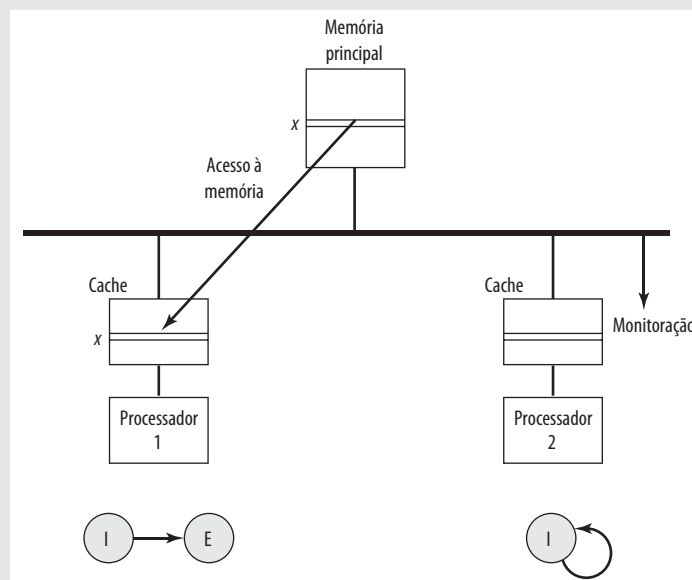
Perguntas de revisão

- 17.1 Relacione e defina brevemente três tipos de organização de sistemas computacionais.
- 17.2 Quais são as principais características de um SMP?
- 17.3 Quais são algumas vantagens potenciais de um SMP comparado com um uniprocessador?
- 17.4 Quais são algumas das principais questões a respeito de projeto de um sistema operacional para um SMP?
- 17.5 Qual é a diferença entre esquemas de coerência de cache por software e por hardware?
- 17.6 Qual é o significado de cada um dos quatro estados no protocolo MESI?
- 17.7 Quais são alguns dos principais benefícios de *clusters*?
- 17.8 Qual é a diferença entre *failover* e *failback*?
- 17.9 Quais são as diferenças entre UMA, NUMA e CC-NUMA?

Problemas

- 17.1** Seja α a percentagem do código do programa que pode ser executado simultaneamente por n processadores em um sistema de computação. Suponha que o código restante deve ser executado sequencialmente por um único processador. Cada processador tem uma taxa de execução de x MIPS.
- Derive uma expressão para taxa MIPS efetiva quando usado o sistema para execução exclusiva deste programa, em termos de n , α e x .
 - Se $n = 16$ e $x = 4$ MIPS, determine o valor de α que produzirá um desempenho de sistema de 40 MIPS.
- 17.2** Um multiprocessador com oito processadores possui 20 unidades de fitas anexados. Há um grande número de trabalhos submetidos ao sistema onde cada um deles requer um máximo de quatro unidades de fitas para completar a execução. Suponha que cada trabalho inicie a execução com apenas três unidades de fitas por um período longo antes de requerer a quarta fita por um período curto próximo do fim da execução. Suponha também um fornecimento interminável desses trabalhos.
- Suponha que o agendador do SO não iniciará um trabalho sem que haja quatro unidades de fitas disponíveis. Quando um trabalho é iniciado, quatro unidades são atribuídos imediatamente e não são liberados até que o trabalho termine. Qual é o número máximo de trabalhos que podem estar em progresso ao mesmo tempo? Quais são os números mínimo e máximo de drives de fita que podem estar ociosos como resultado desta estratégia?
 - Sugira uma política alternativa para melhorar a utilização da unidade de fita e, ao mesmo tempo, evitar *deadlock* de sistema. Qual é o número máximo de trabalhos que pode estar em progresso ao mesmo tempo? Quais são os limites do número de fitas ociosas.
- 17.3** Você consegue ver algum problema com a abordagem de cache escrever-uma-vez (*write once*) em multiprocessadores baseados em barramento? Se sim, sugira uma solução.
- 17.4** Considere uma situação onde dois processadores em uma configuração SMP, ao longo do tempo, requerem acesso à mesma linha de dados da memória principal. Ambos possuem cache e usam protocolo MESI. Inicialmente, as duas caches possuem uma cópia inválida da linha. A Figura 17.22 ilustra a consequência de uma leitura da linha x pelo processador P1. Se este for o começo da sequência de acessos, desenhe as figuras subsequentes para a seguinte sequência:
- P2 lê x .
 - P1 escreve em x (para ficar mais claro, marque a linha na cache do P1 como x').
 - P1 escreve em x (marque a linha na cache do P1 como x'').
 - P2 lê x .

Figura 17.22 Exemplo MESI: Processador 1 lê a linha x



- 17.5** A Figura 17.23 mostra um diagrama de estados de dois protocolos possíveis para coerência de cache. Deduza e explique cada protocolo e compare-os com MESI.
- 17.6** Considere um SMP com caches L1 e L2 usando protocolo MESI. Conforme explicado na Seção 17.3, um dos quatro estados é associado com cada linha da cache L2. Todos os quatro estados também são necessários para cada linha da cache L1? Se sim, por quê? Se não, explique quais estados podem ser excluídos.
- 17.7** Uma versão anterior do mainframe da IBM, S/390 G4, usava três níveis de cache. Assim como no z990, apenas o primeiro nível estava no chip do processador, chamado de unidade de processamento (PU). A cache L2 também era parecida com o z990. Uma cache L3 estava em um chip separado que agia como um controlador de memória e estava interposto entre as caches L2 e cartões de memória. A Tabela 17.4 mostra o desempenho de um arranjo de cache em três níveis para o IBM S/390. O propósito deste problema é determinar se a inclusão de um terceiro nível de cache vale à pena. Determine a penalidade de acesso (número médio de ciclos de CPU) para um sistema com apenas uma cache L1 e normalize esse valor para 1.0. Determine então a penalidade de acesso normalizado quando caches L1 e L2 são usadas e a penalidade de acesso quando todas as três caches são usadas. Observe a quantidade de melhoria em cada caso e dê a sua opinião sobre o valor da cache L3.
- 17.8**
- a. Considere um uniprocessador com caches separadas de dados e instruções, com taxa de acerto H_d e H_i , respectivamente. O tempo de acesso do processador à cache é de c ciclos de clock e tempo de transferência para um bloco entre memória e cache é de b ciclos de clock. Seja f_i a fração dos acessos à memória que são para as instruções e seja f_d a fração de linhas sujas na cache de dados entre linhas substituídas. Suponha uma política de write-back e determine o tempo efetivo de acesso à memória em termos dos parâmetros que acabamos de definir.

Figura 17.23 Protocolos de coerência de duas caches

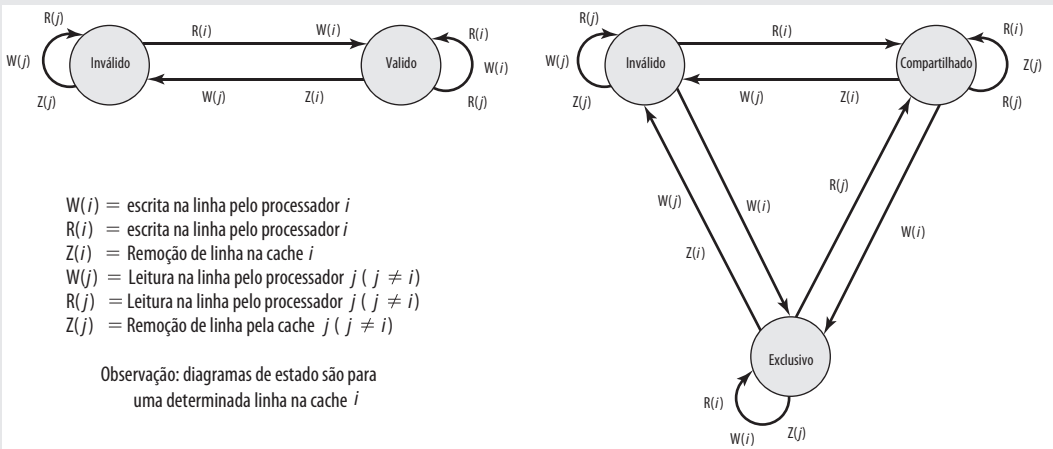


Tabela 17.4 Taxa de sucesso de cache típica na configuração S/390 SMP [MAK97]

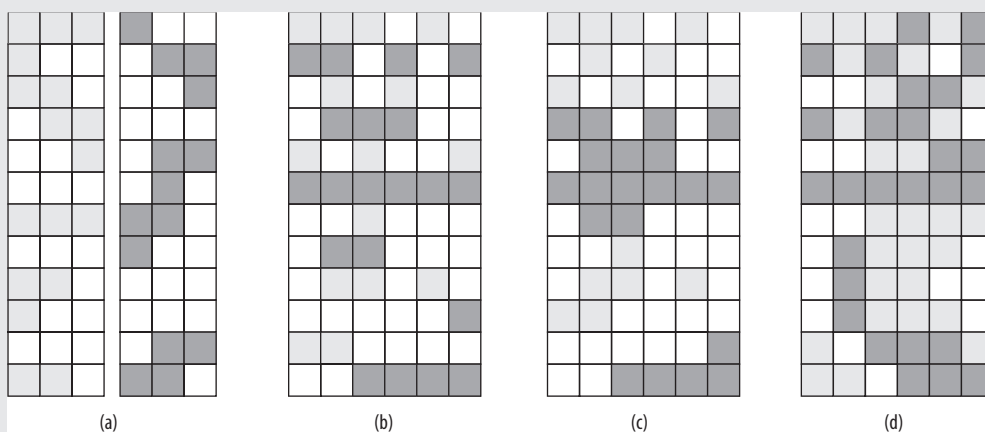
Subsistema de memória	Penalidade de acesso (ciclos de PU)	Tamanho de cache	Taxa de acerto (hit) (%)
Cache L1	1	32 KB	89
Cache L2	5	256 KB	5
Cache L3	14	2 MB	3
Memória	32	8 GB	3

- b. Suponha agora um SMP baseado em barramento onde cada processador tem características da parte (a). Cada processador deve lidar com invalidação de cache além das leituras e escritas de memória. Isto afeta o tempo efetivo de acesso à memória. Seja f_{inv} a fração de referências de dados que fazem com que sinais de invalidação sejam enviados para outras caches de dados. Para o processador enviar sinais, são requeridos t ciclos de clock para completar a operação da invalidação. Outros processadores não são envolvidos na operação da invalidação. Determine o tempo efetivo de acesso à memória.
- 17.9** Qual alternativa organizacional é sugerida por cada uma das ilustrações na Figura 17.24?
- 17.10** Na Figura 17.8, alguns dos diagramas mostram linhas horizontais preenchidas parcialmente. Em outros casos, há linhas totalmente vazias. Isto representa dois tipos diferentes de perda da eficiência. Explique.
- 17.11** Considere o desenho do pipeline na Figura 12.13b, o qual é redesenhado na Figura 17.25a, onde os estágios de busca e decodificação são ignorados, para representar a execução de thread A. A Figura 17.25 ilustra a execução de uma thread B separada. Em ambos os casos, um processador com pipeline simples é usado.
- Mostre um diagrama de envio de instruções, semelhante à Figura 17.8a, para cada uma das duas threads.
 - Suponha que duas threads estão para ser executadas em paralelo em um chip multiprocessador, onde cada um dos dois processadores do chip usam um pipeline simples. Mostre um diagrama de emissão de instruções semelhante à Figura 17.8k. Mostre também um diagrama de execução de pipeline no estilo da Figura 17.25.
 - Suponha uma arquitetura superescalar de envio dupla. Repita a parte (b) para uma implementação superescalar multithread intercalada, supondo que não haja nenhuma dependência de dados. *Observação:* não há uma resposta única; você precisa fazer suposições a respeito de latências e prioridades.
 - Repita a parte (c) para uma implementação superescalar *multithread* bloqueada.
 - Repita para uma arquitetura SMT com quatro envios.
- 17.12** O seguinte segmento de código precisa ser executado 64 vezes para avaliar a expressão aritmética vetorial: $D(I) = A(I) + B(I) \times C(I)$ para $0 \leq I \leq 63$.
- ```

Load R1, B(I) /R1 ← Memory (α + I) /
Load R2, C(I) /R2 ← Memory (β + I) /
Multiply R1, R2 /R1 ← (R1) × (R2) /
Load R3, A(I) /R3 ← Memory (γ + I) /
Add R3, R1 /R3 ← (R3) + (R1) /
Load D1, R3 /Memory (θ + I) ← (R3) /

```
- onde R1, R2 e R3 são registradores do processador, e  $\alpha, \beta, \gamma, \theta$  são os endereços iniciais de memória dos vetores B(I), C(I), A(I) e D(I), respectivamente. Suponha quatro ciclos para cada *Load* ou *Store*, dois ciclos para *Add* e oito ciclos para *Multiply* em um uniprocessador ou em um único processador em uma máquina SIMD.
- Calcule o número total de ciclos de processador necessários para executar este segmento de código repetidamente 64 vezes em um uniprocessador SISD sequencialmente, ignorando todos os outros atrasos de tempo.

**Figura 17.24** Diagrama para Problema 17.9



**Figura 17.25** Duas threads de execução

|    | C0  | F0  | E1  | W0  |    | C0 | F0 | E1 | W0 |
|----|-----|-----|-----|-----|----|----|----|----|----|
| 1  | A1  |     |     |     | 1  | B1 |    |    |    |
| 2  | A2  | A1  |     |     | 2  | B2 | B1 |    |    |
| 3  | A3  | A2  | A1  |     | 3  | B3 | B2 | B1 |    |
| 4  | A4  | A3  | A2  | A1  | 4  | B4 | B3 | B2 | B1 |
| 5  | A5  | A4  | A3  | A2  | 5  |    |    | B3 | B2 |
| 6  |     |     |     | A3  | 6  |    |    |    | B3 |
| 7  |     |     |     |     | 7  | B5 | B4 |    |    |
| 8  | A15 |     |     |     | 8  | B6 | B5 | B4 |    |
| 9  | A16 | A15 |     |     | 9  | B7 | B6 | B5 | B4 |
| 10 |     | A16 | A15 |     | 10 |    | B7 | B6 | B5 |
| 11 |     |     | A16 | A15 | 11 |    |    | B7 | B6 |
| 12 |     |     |     | A16 | 12 |    |    |    | B7 |

(a) (b)

- b. Considere o uso de um computador SIMD com 64 elementos de processamento para executar operações vetoriais em seis instruções vetoriais sincronizadas em cima de um vetor de 64 componentes de dados e todos conduzidos por uma mesma velocidade de clock. Calcule o tempo total de execução na máquina SIMD, ignorando broadcast (difusão) de instruções e outros atrasos.
- c. Qual o aumento de velocidade ganho pelo computador SIMD em relação ao computador SISD?

**17.13** Produza uma versão vetorial do seguinte programa: **DO** 20 I = 1, N

```

 B(I, 1) = 0
 DO 10 J = 1, M
 A(I) = A(I) + B(I, J) × C(I, J)
10 CONTINUE
 D(I) = E(I) + A(I)
20 CONTINUE

```

**17.14** Uma aplicação é executada em um *cluster* de nove computadores. Um programa que mede desempenho levou tempo  $T$  neste *cluster*. Depois foi encontrado que 25% de  $T$  foi o tempo durante o qual a aplicação estava executando simultaneamente em todos os nove computadores. No tempo restante, a aplicação teve que executar em um único computador.

- a. Calcule o aumento efetivo de velocidade sob a condição anterior quando comparado à execução do programa em um único computador. Calcule também  $\alpha$ , a percentagem de código que foi paralelizada (programada ou compilada de tal forma que utilize o modo de *cluster*) no programa anterior.
- b. Suponha que somos capazes de usar efetivamente 17 computadores em vez de 9 na parte paralelizada do código. Calcule o aumento de velocidade efetivo que é alcançado.

**17.15** O seguinte programa FORTRAN está para ser executado em um computador e uma versão paralelizada está para ser executada em um *cluster* de 32 computadores.

```

L1: DO 10 I = 1, 1024
L2: SUM(I) = 0
L3: DO 20 J = 1, I
L4: 20 SUM(I) = SUM(I) + I
L5: 10 CONTINUE

```

Suponha que as linhas 2 e 4 levem, cada uma, dois ciclos de máquina, incluindo todas as atividades do processador e acesso à memória. Ignore o *overhead* causado pelo controle do software sobre laços (linhas 1, 3, 5) e todas as outras sobrecargas do sistema e conflitos de recursos.

- Qual o tempo total de execução (em número de ciclos de máquina) do programa em um único computador?
- Divida as iterações do laço  $I$  entre 32 computadores da seguinte forma: computador 1 executa as primeiras 32 iterações ( $I = 1$  até 32), processador 2 executa próximas 32 alterações, e assim por diante. Quais são os fatores de aumento de velocidade e tempo de execução quando comparado com parte (a)? (Observe que a carga computacional, ditada pelo laço  $J$ , não está equilibrada entre os computadores.)
- Explique como modificar o paralelismo para facilitar uma execução paralela balanceada de toda a carga computacional através de 32 computadores. Uma carga balanceada significa aqui que um número igual de adições é atribuído para cada computador com respeito a ambos os laços.
- Qual o tempo mínimo de execução resultante da execução paralela em 32 computadores? Qual o aumento de velocidade resultante em relação a um único computador?

**17.16** Considere duas versões de um programa para somar dois vetores:

|                        |                                 |
|------------------------|---------------------------------|
| L1: DO 10 I = 1, N     | DOALL K = 1, M                  |
| L2: A(I) = B(I) + C(I) | DO 10 I = L(K-1) + 1, KL        |
| L3: 10 CONTINUE        | A(I) = B(I) + C(I)              |
| L4: SUM = 0            | 10 CONTINUE                     |
| L5: DO 20 J = 1, N     | SUM(K) = 0                      |
| L6: SUM = SUM + A(J)   | DO 20 J = 1, L                  |
| L7: 20 CONTINUE        | SUM(K) = SUM(K) + A(L(K-1) + J) |
|                        | 20 CONTINUE                     |
|                        | ENDALL                          |

- O programa da esquerda executa em um uniprocessador. Suponha que cada linha de código L2, L4 e L6 leve um ciclo de clock do processador para executar. Para simplicidade, ignore o tempo necessário para outras linhas de código. Inicialmente todas as matrizes já estão carregadas na memória principal e o pequeno pedaço do programa está na cache de instruções. Quantos ciclos de clock são necessários para executar este programa?
- O programa da direita é escrito para executar em um multiprocessador com  $M$  processadores. Particionamos as operações de iteração em seções  $M$  com  $L = N/M$  elementos por seção. DOALL declara que todas as seções  $M$  são executadas em paralelo. O resultado deste programa é produzir  $M$  somas parciais. Suponha que  $k$  ciclos de clock são necessários para cada operação de comunicação entre processadores através da memória compartilhada e que, por isso, a adição de cada soma parcial requer  $k$  ciclos. Uma árvore binária de soma de  $I$  níveis pode juntar todas as somas parciais, onde  $I = \log_2 M$ . Quantos ciclos são necessários para produzir a soma final?
- Suponha  $N = 2^{20}$  elementos na matriz e  $M = 256$ . Qual o aumento de velocidade obtido com uso do multiprocessador? Suponha  $k = 200$ . Qual percentagem é esta do aumento teórico de velocidade de um fator de 256?

## Referências

- FLYNN, M. "Some computer organizations and their effectiveness". *IEEE Transactions on Computers*, set. 1972.
- SIEGEL, T.; PFEFFER, E. e MAGEE, A. "The IBM z990 microprocessor". *IBM Journal of Research and Development*, maio/jul. 2004.
- MAK, P., et al. "Processor subsystem interconnect for a large symmetric multiprocessing system". *IBM Journal of Research and Development*, mai./jul. 2004.
- LILJA, D. "Cache coherence in large-scale shared-memory multiprocessors: issues and comparisons". *ACM Computing Surveys*, set. 1993.
- STENSTROM, P. "A survey of cache coherence schemes of multiprocessors". *Computer*, jun. 1990.
- SHANLEY, T. *Unabridged Pentium 4, the: IA32 processor genealogy*. Reading, MA: Addison-Wesley, 2005.
- UNGERER, T.; RUBIC, B. e SILC, J. "Multithreaded Processors". *The Computer Journal*, No. 3, 2002.
- UNGERER, T.; RUBIC, B.; e SILC, J. "A survey of processors with explicit multithreading". *ACM Computing Surveys*, mar. 2003.
- MARR, D., et al. "Hyper-threading technology architecture and microarchitecture". *Intel Technology Journal*, primeiro trimestre de 2002.
- KALLA, R.; SINHARROY, B. e TENDLER, J. "IBM Power5 chip: a dual-core multithreaded processor". *IEEE Micro*, mar./abr. 2004.
- BREWER, E. "Clustering: multiply and conquer". *Data Communications*, jul. 1997.
- KAPP, C. "Managing cluster computers". *Dr. Dobbs Journal*, jul. 2000.
- HWANG, K., et al. "Designing SSI clusters with hierarchical checkpointing and single I/O space". *IEEE Concurrency*, jan./mar. 1999.
- BUYIA, R. *High performance cluster computing: architectures and systems*. Upper Saddle River, NJ: Prentice Hall. 1999.
- NOWELL, M.; VUSIRIKALA, V. e HAYS, R. "Overview of requirements and applications for 40 gigabit and 100 gigabit ethernet". *Ethernet Alliance White Paper*, ago. 2007.

- p WHITNEY, S., et al. "The SGI origin software environment and application performance". *Proceedings, COMPCON Spring '97*, fev. 1997.
- q LOVETT, T. e CLAPP, R. "Implementation and performance of a CC-Numa system". *Proceedings, 23rd Annual International Symposium on Computer Architecture*, mai. 1996.
- r PFISTER, G. *In search of clusters*. Upper Saddle River, NJ: Prentice Hall, 1998.
- s PADEGS, A. et al. "The IBM System/370 vector architecture: design considerations". *IEEE Transactions on Communications*, mai. 1988.
- t TUCKER, S. "The IBM 3090 System design with emphasis on the vector facility". *Proceedings, COMPCON Spring '87*, fev. 1987.
- u CATANZARO, B. *Multiprocessor system architectures*. Mountain View, CA: Sunsoft Press, 1994.
- v STONE, H. *High-performance computer architecture*. Reading, MA: Addison-Wesley, 1993.
- w HWANG, K. *Advanced computer architecture*. Nova York: McGraw-Hill, 1993.
- x MILENKOVIC, A. "Achieving high performance in bus-based shared-memory multiprocessors". *IEEE Concurrency*, jul./set. 2000.
- y TOMASEVIC, M. e MILUTINOVIC, V. *The cache coherence problem in shared-memory multiprocessors: hardware solutions*. Los Alamitos, CA: IEEE Computer Society Press, 1993.
- z BUYYA, R. *High performance cluster computing: programming and applications*. Upper Saddle River, NJ: Prentice Hall. 1999.
- aa WEYGANT, P. *Clusters for high availability*. Upper Saddle River, NJ: Prentice Hall, 2001.
- bb DESAI, D., et al. "BladeCenter system overview". *IBM Journal of Research and Development*, nov. 2005.